Concepts Shared by All Predictors: Colloquium Textbook

Thesis: A Unified Benchmark of Grokking Predictors in Neural Networks · Common concepts (not predictor-specific) · Prepared 5 November 2026

This is the study book for the concepts shared by every predictor in your benchmark: 7 modules and 29 topics, in the order you'd meet them, from grokking itself to research honesty. It doesn't describe any single predictor; those come later, in their own books. Every event list used in a worked example is made up for illustration and belongs to no predictor of yours. Every term is defined in the Notation and glossary chapter before it's used.

Every topic is laid out the same way. **In plain words** says what the idea is, why it's in your thesis, and what mistake it prevents, in everyday language. **Definition** then states it exactly. **Why it matters and how to read it** explains how to think about the definition and what to look for. The **Worked example** does the maths in H.K. Dass layout, one step per line, and **Reading the answer** says what the result means. Each topic ends with a **Self-check** (answers in brackets) and a **Likely viva question**. You should be able to read each topic on its own, without asking anyone what a sentence means.

Jonathan John · M.Sc. AI

Contents

Notation and glossary **2**

1 The grokking phenomenon **8**

1.1 Grokking 8

1.2 Memorisation vs generalisation 9

1.3 Circuit formation and cleanup 10

2 Your training setup **12**

2.1 Modular addition task 12

2.2 Train/test split (30%) 13

2.3 Transformer, logits and cross-entropy loss 15

2.4 AdamW and weight decay 17

2.5 Epochs and checkpoints 19

2.6 Seeds and why five 20

3 Measuring the outcome **22**

3.1 Test accuracy 22

3.2 The grok epoch G_i 23

4 What a predictor is **25**

4.1 Predictor and event epoch P_i 25

4.2 Lead and the 100-epoch margin 26

4.3 Lead fraction 27

4.4 Relative lead and mean relative lead 29

4.5 Misses and `None` 31

5 Spearman's ρ and small samples **33**

5.1 Spearman rank correlation ρ 33

5.2 Pearson vs Spearman 35

5.3 Only 21 possible values with 5 seeds 37

5.4 Significance with small n 38

6 The frozen protocol and the PASS rule **41**

6.1 The frozen protocol 41

6.2 The PASS rule 42

6.3 Pre-registration and criterion validation 44

7 Research honesty **46**

7.1 Hindsight (look-ahead) bias 46

7.2 Causal vs centred moving average 47

7.3 Researcher degrees of freedom 48

7.4 Sensitivity analysis 50

7.5 Baselines, controls and noise floor 51

7.6 Negative results and reproducibility 52

Recommended reading **53**

Notation and glossary

Every symbol and term in this book is defined here, once, in dependency order: each entry uses only entries above it. The modules use only these names. Entries are numbered **A1**, **N1**, **N2**, … so the text can point back to them. For each entry you get what it is, its type, where it's evaluated, its sign or direction where that matters, and its role in the benchmark. Read it once from start to finish before Module 1; after that, come back to an entry whenever the text points to it. The modules explain each idea again in full sentences, so the glossary is a reference, not something to learn by heart.

Part A. How to read the maths

A1 Set, $\in$, #. A set is a collection of objects written in braces, e.g. $\{0, 1, 2\}$. $x \in S$ means "x belongs to the set S". $\#S$ is an integer: the number of objects in S.

A2 $\min\{\ldots\}$**.** The smallest number in a set of numbers.

A3 $\lfloor x \rfloor$ **(floor).** For a scalar $x \geq 0$, the largest integer that is $\leq x$. Python's `int()` gives the same result for $x \geq 0$.

A4 mod. For whole numbers $x \geq 0$ and $y > 0$, x mod y is the remainder when x is divided by y: a whole number from 0 to $y - 1$. For example, 150 mod $113 = 37$. (x and y are placeholders in this entry only.)

A5 $=$ **and** $\approx$**.** $=$ means exactly equal. $\approx$ means approximately equal. In a derivation $\approx$ appears once, on the line where digits or terms are dropped, and that line says what was dropped.

A6 $\because$ **and** $\therefore$**.** $\because$ means "because"; every line of a solution ends with one. $\therefore$ means "therefore"; it opens the conclusion.

A7 Derivation layout (H.K. Dass). *Given*: a numbered list of facts, each naming where it comes from (a glossary entry, a section, or a file of yours). *Let*: each symbol used, defined once. *To find*: what is computed or shown. *Solution*: one aligned block, one step per line. Then a $\therefore$ sentence in plain words and a boxed answer.

A8 d.p. Decimal places. "0.991 (3 d.p.)" means the value rounded to 3 decimal places.

A9 Your files. `nanda_unified.yaml`: the configuration (config) file of the final setup. `analyze_nanda_unified.py`: the analysis script. `summary.json`: the saved results file. `context.md`: your project log.

A10 Citations [1]–[5]. Numbers in square brackets point to the Recommended reading list at the end. "Power et al." is [1]; "Nanda et al." or "Nanda" is [2].

A11 Viva, examiner, attack point, audit finding. The viva is your oral exam (colloquium). The examiner asks the questions. An attack point is a weakness in your work that an examiner could raise there. An audit finding is an issue found in the 3 November 2026 audit of your code.

Part B. The task and the data

N1 p**, the modulus.** Integer, prime, $p = 113$. Set in `nanda_unified.yaml` and never changed. Role: sets the size of the task.

N2 a, b**, the inputs.** Two integers, each in $\{0, 1, \ldots, p-1\} = \{0, \ldots, 112\}$. Role: the two numbers the model reads.

N3 c**, the correct answer.** Integer, $c = (a + b) \bmod p$, so $c \in \{0, \ldots, 112\}$. Role: the label the model must output.

N4 Pair and N**.** A pair is one example: the inputs (a, b) together with their answer c. N is an integer, the number of all pairs: $N = p^2 = 12{,}769$ (worked out in Section 2.1).

N5 The model, its weights θ**, and training.** The model is a one-layer, four-head transformer (a standard type of neural network; "one-layer, four-head" names its size, and its insides aren't needed here). It's a function: it reads a pair's inputs (a, b) and a weight vector θ, and returns 113 scores, one per possible answer. θ is a vector with M entries $\theta_1, \ldots, \theta_M$: every trainable number of the model. M is a positive integer whose value this book doesn't need. Training means changing θ again and again so that the model's answers on training pairs get better. Nanda's model also has a weight matrix called W_O (the attention output matrix); yours doesn't. That's a known gap.

N6 f**, the train fraction.** Scalar, $f = 0.3$. It's hard-coded in `get_dataloaders` as `train_size = int(0.3 * len(...))`; the config entry `train_fraction` doesn't change it. Role: the share of pairs used for training.

N7 D_{train}**,** D_{test}**,** N_{train}**,** N_{test}**.** D_{train} is the training set: a set of $N_{\text{train}} = 3{,}830$ pairs that are used to change the weights. D_{test} is the test set: the other $N_{\text{test}} = 8{,}939$ pairs, never used to change the weights. N_{train} and N_{test} are integers (worked out in Section 2.2). Role: D_{test} measures whether the model has learned the rule.

N8 Seed i **and run.** A seed is an integer label $i \in \{0, 1, 2, 3, 4\}$ that fixes the random numbers of one run (the initial weights and the data split). A run is one full training of the model with one seed. Role: 5 seeds give 5 independent, repeatable runs.

Part C. The model and its training

N9 Epoch t **and full batch.** An epoch is one full pass over D_{train}. t is an integer epoch counter, $t \in \{0, 1, \ldots, 40{,}000\}$; $t = 0$ means before any training. Every run lasts 40,000 epochs. Your runs are full batch: all 3,830 training pairs are used together each time θ is changed, so θ changes once per epoch.

N10 θ_t**, the weights after epoch** t**.** Vector with M entries, $\theta_t = (\theta_{t,1}, \ldots, \theta_{t,M})$: the value of θ (**N5**) at the end of epoch t. θ_0 is the initial weights. Role: everything the model computes at epoch t comes from θ_t.

N11 z**, the logits.** Vector with $p = 113$ entries $z_0, \ldots, z_{112}$, computed by the model for one pair at θ_t. Entry z_k is the score for answer $k \in \{0, \ldots, 112\}$; a larger z_k means the model favours k more.

N12 $\hat{k}$**, the predicted answer.** Integer in $\{0, \ldots, 112\}$: the k with the largest z_k, at θ_t.

The model is correct on a pair when $\hat{k} = c$.

N13 q_k**, the softmax probability.** Scalar in $(0, 1)$, computed from z at θ_t: $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$, where j is a summation index over the 113 answers. The 113 values q_k add up to 1. q_c is the probability the model gives the correct answer. (The letter q is used so it doesn't clash with the modulus p.)

N14 ℓ**, the loss of one pair (cross-entropy).** Scalar, $\ell = -\ln q_c$, at θ_t. Sign: $\ell \geq 0$; $\ell = 0$ when $q_c = 1$, and ℓ grows as q_c falls. Role: punishes a low probability on the correct answer.

N15 $L(\theta_t)$**, the training loss.** Scalar: the average of ℓ over the 3,830 pairs of D_{train}, at θ_t. Role: the number training tries to make small.

N16 $g(\theta_t)$**, the gradient.** Vector with M entries; entry m is $\partial L / \partial \theta_m$, the partial derivative (how fast L changes when only θ_m changes), evaluated at θ_t. Direction: $g(\theta_t)$ points uphill (the direction in which L grows fastest), so a step along $-g(\theta_t)$ goes downhill.

N17 η**, the learning rate.** Scalar, $\eta > 0$, the same at every epoch. Role: the step size of each update. Its value is in the config; this book doesn't need it.

N18 AdamW, β_1**,** β_2**,** u_t**.** AdamW is the optimiser: the method that turns gradients into weight updates [3]. It keeps running averages of the gradient and of the squared gradient; $\beta_1 = 0.9$ and $\beta_2 = 0.98$ are scalars that set how much of the past those averages keep. u_t is the AdamW step: a vector with M entries, computed at epoch t from $g(\theta_0), \ldots, g(\theta_t)$ and β_1, β_2. Direction: like g, it points roughly uphill, so $-\eta u_t$ is a step downhill.

N19 λ**, the weight decay.** Scalar, $\lambda = 1.0$, the same at every epoch. Sign: $\lambda \geq 0$. Role: shrinks every weight a little at each update.

N20 The weight update, decoupled, confound. $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$. The middle term is the AdamW step downhill. The last term is the decoupled weight decay: decoupled means it's applied directly to θ_t, separately from u_t. It pulls every weight towards 0 (Section 2.4). A confound is a second cause that can produce the same effect as the one you're testing; weight decay is one for any quantity computed from the size of the weights.

N21 Checkpoint. A saved copy of θ_t at one epoch t. Checkpoints are saved every 50 epochs and evaluated every 100 epochs; evaluating a checkpoint means computing from it the quantities that need saved weights. Role: some quantities are computed afterwards from checkpoints.

Part D. The outcome

N22 $\text{acc}_{\text{train}}(t)$**,** $\text{acc}_{\text{test}}(t)$**.** Scalars in $[0, 1]$: the fraction of pairs in D_{train} (or D_{test}) on which $\hat{k} = c$, using θ_t. Direction: higher is better. Role: acc_{test} measures how well the model does on pairs it was never trained on.

N23 Memorisation and generalisation. Memorisation: getting the training pairs right by storing them, so $\text{acc}_{\text{train}}(t)$ is high while $\text{acc}_{\text{test}}(t)$ stays low. Generalisation: getting new pairs right because the rule $(a + b) \bmod p$ has been learned, so $\text{acc}_{\text{test}}(t)$ is high too.

N24 Grokking. In one run, $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later $\text{acc}_{\text{test}}(t)$

suddenly jumps to high values. It is a switch from memorisation to generalisation.

N25 Circuit and Nanda's three phases. A circuit is a set of weights in θ_t that together compute the general rule $(a+b)$ mod p. Nanda's three phases [2]: memorisation; circuit formation (the circuit is built while acc_{test} stays low); cleanup (the weights used for memorising are removed, and acc_{test} jumps).

N26 Grok threshold and grok epoch G_i. The grok threshold is the scalar 0.9 (the first entry of `grok_thresholds` in the config). G_i is an integer epoch: $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ in the run of seed i. So $\text{acc}_{\text{test}}(t) \leq 0.9$ at every epoch $t < G_i$. Values: $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. Role: the epoch the benchmark tries to warn about in advance. "Grok" on its own ("before grok") means the grok epoch G_i of that run. The G_i are an outcome of training: they don't depend on how any warning is computed.

Part E. Predictors and events

N27 Signal y_t, y_{final}. A signal is a scalar computed at epoch t during training, from θ_t or from the model's outputs. Example: the training loss $L(\theta_t)$ (**N15**) is a signal. y_{final} is its value at the last epoch, $t = 40{,}000$.

N28 Moving average, window w, causal, centred, ex-ante. A moving average replaces each point of a signal curve by the average of w neighbouring points; w is a positive integer, the window. A causal moving average at epoch t uses only points at or before t. A centred one uses points on both sides of t, so it uses later points too. `uniform_filter1d` (a smoothing function available in your code) is centred. An ex-ante trigger is a rule that uses only the past.

N29 Rule, fire, event epoch P_i, miss, `None`. A rule (event rule) is a fixed procedure that reads a signal curve and names one epoch. If it names an epoch in the run of seed i, the rule fires, and that epoch is the event epoch P_i: a scalar epoch (it need not be a whole number). If it never names an epoch, that's a miss, and the code stores `None` instead of a number.

N30 Rule settings and `skip_epochs`. The fixed numbers a rule uses, such as a window w (**N28**) or a threshold level. `skip_epochs` is one such integer setting: the number of first epochs a rule ignores. Changing any setting changes the rule.

N31 Predictor. A (signal, rule) pair. If the rule or one of its settings changes, it's a different predictor. Your benchmark has eight predictors. This book names none of them: everything in it applies to all eight.

N32 Illustrative event list. A made-up list of event epochs $(P_0, \ldots, P_4)$, one per seed, written down only to show how a calculation works. It isn't the output of any of your predictors. The book introduces List A (Section 4.1), List B (Section 5.2) and List C (Section 6.2), plus a few one-line lists inside single sections.

N33 Firing set F and n. F is the set of seeds whose rule fired: $F \subseteq \{0, 1, 2, 3, 4\}$. $n = \#F$ is an integer from 0 to 5. Seeds with `None` are dropped before anything is computed. If $n < 4$, the predictor is rejected automatically, whatever its other numbers are.

N34 Lead and the 100-epoch margin. Seed $i \in F$ leads if $P_i < G_i - 100$: the event comes more than 100 epochs before grok. 100 is the margin, an integer number

of epochs.

N35 The `seeds_before_grok` count. A count in your saved results that calls a seed early if $G_i - P_i > 0$, not if $P_i < G_i - 100$. So it ignores the margin of **N34**. That's an audit finding.

N36 lead_frac**, the lead fraction.** Scalar in $[0, 1]$: lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$, the share of firing seeds that lead.

N37 r_i **and** $\bar{r}$**, relative lead.** $r_i = (G_i - P_i)/G_i$ is a scalar for seed $i \in F$. Sign: $r_i > 0$ when the event is before grok, $r_i < 0$ when after; r_i near 1 means the event is very early compared with grok. $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$ is the mean relative lead, a scalar.

N38 Rank $R(X_i)$ **and ties.** For a list of n numbers X_i, $R(X_i)$ is the position of X_i when the list is sorted from smallest to largest; the smallest has rank 1. A tie is two or more equal values; tied values all get the average of the positions they occupy.

N39 Mean $\bar{X}$ **and Pearson correlation** $C(X, Y)$**.** For two lists X_i, Y_i of scalars over the same n seeds, $\bar{X} = \frac{1}{n}\sum X_i$ (a scalar, the mean), and $C(X, Y) = \dfrac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}}$, a scalar in $[-1, 1]$. It measures how close the points are to a straight line; $+1$ means a rising line. It is undefined if either list is constant (the denominator is 0); code then typically returns NaN, "not a number".

N40 Spearman's ρ**,** d_i**, fixed clock.** Scalar in $[-1, 1]$: $\rho = C\big(R(P), R(G)\big)$, the Pearson correlation of the ranks, over the n seeds in F. $+1$ means later grok always comes with a later event. With no ties it equals $\rho = 1 - \dfrac{6\sum_{i \in F} d_i^2}{n(n^2 - 1)}$, where $d_i = R(G_i) - R(P_i)$ is an integer, the rank difference of seed i. A fixed clock is an event at about the same epoch in every run, whatever G_i is; its ρ is undefined (all events tied) or set by luck, by the order of tiny differences.

N41 Adjacent swap. Two seeds whose ranks are next to each other (e.g. 2 and 3) appear in the opposite order in P and in G.

N42 $n!$**, chance and p-value.** $n! = n \times (n - 1) \times \cdots \times 1$ is the number of orderings of n items. "By chance" means all $n!$ orderings of the events are equally likely. The p-value of a ρ is the probability, by chance, of a ρ at least that large. A ρ is called significant when its p-value is small.

Part F. Judging predictors, and research honesty

N43 PASS rule, FAIL, verdict. A predictor gets the verdict PASS only if all three hold: lead_frac ≥ 0.8, $\rho \geq 0.9$ and $0.05 \leq \bar{r} \leq 0.9$. Otherwise its verdict is FAIL. A predictor rejected automatically for $n < 4$ (**N33**) also gets FAIL. The function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies this rule; scoring a predictor means running it. In the code the ρ cutoff is written as 0.8999999999999998 so that a ρ of 0.9 isn't lost to floating-point rounding.

N44 Frozen protocol. The fixed set of rules used to score every predictor: `nanda_unified.yaml`, $p = 113$, 5 seeds, 40,000 epochs, grok threshold 0.9, and the PASS rule (**N43**). It was written on 29 September 2026 and isn't changed while predictors are scored.

N45 Closed-negative. A predictor whose study is finished with a FAIL verdict. The

first seven of your eight predictors are closed-negative.

N46 Pre-registration, criterion validation, synthetic predictor, calibration, post-hoc. Pre-registration: writing down the test before running it. A synthetic predictor is a made-up list of event epochs built from the known G_i, used to test the test. Criterion validation: checking that the PASS rule passes a good synthetic predictor and fails a bad one. Calibration: setting a threshold of the test using data. Post-hoc: decided after the results were seen.

N47 Look-ahead (hindsight) bias. Using information from later epochs (e.g. y_{final}) to decide when an earlier event happened. A rule free of it could be run live during training.

N48 Researcher degrees of freedom. Choices of rule settings (**N30**), such as windows, thresholds or `skip_epochs`, made after looking at the results.

N49 Sensitivity analysis and near-miss. Sensitivity analysis: changing one setting a little and checking whether the verdict changes. A near-miss is a FAIL that misses a threshold by a small amount.

N50 Baseline (control) and noise floor. A control run is one where grokking doesn't happen, or a setting is switched off (e.g. $\lambda = 0$). The noise floor is how much a signal wobbles for no reason.

N51 Negative result and reproducibility. A negative result is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

Module 1

The grokking phenomenon

This module is about the thing being predicted. Before you can judge any warning, you need to be clear on what it's a warning of. Section 1.1 says what grokking looks like on the two accuracy curves of a run. Section 1.2 names the two states a run passes through, memorisation and generalisation, and shows with your own grok epochs that every run spends thousands of epochs in the first. Section 1.3 gives Nanda's account of what happens inside the weights between the two states, which is the reason anyone expects an early warning to exist at all.

1.1 Grokking

In plain words: Picture the two accuracy curves of one run drawn on the same axes, with epochs along the bottom. The training-accuracy curve climbs to the top almost at once: within a few hundred epochs the model answers every training pair correctly. The test-accuracy curve does something odd. It stays low for thousands of epochs, long after training accuracy has stopped improving, and then it climbs quickly to high values. That late climb is grokking. Your thesis exists because of it: if the climb comes so long after the training curve has settled, you can ask whether anything measurable during training gives warning that it's coming. This section fixes what grokking is, so that you never confuse it with a model that's simply learning slowly.

Definition: Grokking (**N24**) is what happens when, in one run, training accuracy $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later test accuracy $\text{acc}_{\text{test}}(t)$ suddenly jumps to high values.

Why it matters and how to read it: It's the event your whole benchmark tries to predict. Every predictor is judged by whether it signals before the jump, measured by the grok epoch G_i (**N26**).

Two features make grokking different from ordinary learning. The first is the order: training accuracy is high first, and test accuracy is high only much later. The second is the gap: for a long stretch of epochs the model already gets the training pairs right, so the training-accuracy curve shows no sign that anything is still changing, yet test accuracy is still going to jump. That's why you always need both curves. Training accuracy on its own can't show grokking, because it's already about 1 before anything interesting happens. Test accuracy on its own can't show it either, because without the training curve you couldn't tell a late jump after training has finished from a slow climb while training is still going on.

Worked example: In your runs, $\text{acc}_{\text{train}}(t)$ reaches 1 within a few hundred epochs, but $\text{acc}_{\text{test}}(t)$ only passes 0.9 at epochs 6921 to 23747, depending on the seed. There's no calculation here.

Self-check: (1) Why is grokking surprising? (The model already gets every training pair right, so the training-accuracy curve is flat, yet acc_{test} keeps improving thousands of epochs later.) (2) Which two curves do you need to see it? ($\text{acc}_{\text{train}}(t)$ and $\text{acc}_{\text{test}}(t)$

on the same epoch axis: the first shows training has already succeeded, the second shows the late jump.)

Likely viva question: "Define grokking precisely. How is it different from ordinary slow learning?" (In slow learning $\text{acc}_{\text{train}}$ and acc_{test} rise together. In grokking there's a long gap with $\text{acc}_{\text{train}} \approx 1$ and acc_{test} low, then acc_{test} jumps past 0.9 at G_i.)

1.2 Memorisation vs generalisation

In plain words: There are two quite different ways a model can get a training pair right. It can store that pair's answer on its own, with no link to any other pair, or it can have learned the rule that produces every answer, here $(a + b) \bmod p$. On the training pairs you can't tell the two apart: both give the right answer. The test pairs tell them apart, because a stored answer only helps on the pair that was stored, while the rule works on every pair. This section gives the two ways their names, memorisation and generalisation, and shows from your own grok epochs that every run spends thousands of epochs before grok. The mistake it prevents is reading a high training accuracy as proof that the model has learned the task.

Definition: Memorisation (**N23**) is getting the training pairs right by storing them: $\text{acc}_{\text{train}}$ is high while acc_{test} stays low. Generalisation is getting new pairs right because the rule $(a + b) \bmod p$ has been learned: acc_{test} is high too.

Why it matters and how to read it: Grokking is the switch from memorisation to generalisation. So any warning must be about that switch. A signal that only reacts to memorisation tells you nothing about when the switch will come.

This matters as soon as you look at a signal that changes early in training. Early in training every run is memorising, so a signal that changes there may only be reacting to memorisation. Such a signal tells you that memorisation is happening or finishing, which you can already see from $\text{acc}_{\text{train}}$, and nothing about when the switch to generalisation will come. The test that says which state a run is in uses only the two accuracies: $\text{acc}_{\text{train}}$ high and acc_{test} low means memorising; both high means generalising. Because G_i is the first epoch with $\text{acc}_{\text{test}} > 0.9$ (**N26**), any epoch below the smallest G_i is before grok in every run, and you can say so without looking at a single curve. The derivation does exactly that for epoch 1,000.

Worked example: The derivation uses only the five grok epochs to show that, at epoch 1,000, every run is still memorising.

Derivation (H.K. Dass layout): every run is still memorising at epoch 1,000

Given:

1. $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $\text{acc}_{\text{test}}(t) \leq 0.9$ at every epoch $t < G_i$, in the run of seed i (**N26**).
3. $\text{acc}_{\text{train}}(t)$ reaches 1 within a few hundred epochs in every run (Section 1.1).

Let:

1. G_i: the grok epoch of seed i. Integer epoch, positive. Role: when that run starts to generalise.
2. G_{min}: the smallest of the five G_i. Integer epoch. Role: the earliest grok over all runs.
3. $t_1 = 1000$: the epoch being checked. Integer epoch.

To find: Whether $\text{acc}_{\text{test}}(t_1) \leq 0.9$ in every run.

Solution:

$$
\begin{aligned}
G_{\text{min}} &= \min\{12987, 6921, 9510, 11019, 23747\} && \because \text{ definition of } G_{\text{min}} \text{ (Let 2) with Given 1} \\
&= 6921 && \because \text{ seed 1 has the smallest value (\textbf{A2})} \\
t_1 = 1000 &< 6921 = G_{\text{min}} && \because \text{ comparing the two numbers} \\
t_1 &< G_i \text{ for } i = 0, \ldots, 4 && \because \text{ every } G_i \text{ is at least } G_{\text{min}} \\
\text{acc}_{\text{test}}(t_1) &\leq 0.9 \text{ in every run} && \because \text{ Given 2}
\end{aligned}
$$

$\therefore$ At epoch 1,000, acc_{test} is at most 0.9 in all five runs, while $\text{acc}_{\text{train}}$ has already reached 1 (Given 3). Every run is memorising, not yet generalising.

$$\boxed{\text{acc}_{\text{test}}(1000) \leq 0.9 \text{ in all five runs: memorising}}$$

Reading the answer: The calculation needed no curve at all, only the five grok epochs and the definition of G_i. That's the habit to keep: when you're asked which state a run is in at some epoch, compare the epoch with that run's G_i. Different runs can be in different states at the same epoch, because their G_i differ; the self-check below shows one such epoch.

Self-check: At epoch 8,000, which runs have generalised? (Only seed 1: $G_1 = 6921 < 8000$, so its acc_{test} has passed 0.9. Every other G_i is above 8,000, so those four runs are still before grok.)

Likely viva question: "Is your signal detecting the switch to generalisation, or just the end of memorisation?"

1.3 Circuit formation and cleanup

In plain words: Section 1.2 described grokking from the outside, through the two accuracy curves. Nanda et al. [2] looked inside the weights of a model like yours and gave an account of what happens in the long gap between the rise of training accuracy and the jump in test accuracy. Their account has three phases. The reason it matters for your thesis is simple. If the general rule were only built at the moment test accuracy jumps, there would be nothing to detect beforehand, and early warning would be hopeless. Nanda's account says the rule is built gradually, before the jump, while test accuracy still looks flat. That's the premise behind every predictor in your benchmark.

Definition: Nanda et al. [2] describe three phases (**N25**): memorisation; circuit formation, where a circuit that computes $(a+b)$ mod p is built while acc_{test} stays low; and cleanup, where the weights used for memorising are removed and acc_{test} jumps.

Why it matters and how to read it: It explains why a hidden signal could exist before G_i: the circuit is being built before the jump. That's the premise of the whole benchmark.

Read the three phases in order. In memorisation, the weights store the training pairs. In circuit formation, a circuit (**N25**), a set of weights that together compute $(a+b)$ mod p, is built up while the weights used for memorising are still there; the model's answers on test pairs don't yet change much, so acc_{test} stays low. In cleanup, the weights used for memorising are removed, and acc_{test} jumps. The phase a warning would need to detect is circuit formation, because it's the only one that is both before the jump and invisible on the accuracy curves. Keep clear what this account does and doesn't give you. It explains why a hidden signal could exist before G_i. It doesn't say which quantity shows it, or how early; that's exactly what your benchmark tests.

Worked example: Nanda report that cleanup happens at the same time as the jump in acc_{test}, so the circuit already exists before the jump. In your runs that jump is at G_i, between epochs 6921 and 23747. There's no calculation here.

Self-check: Which phase would a good early-warning signal need to detect? (Circuit formation: it comes before cleanup and the jump, and it can't be seen on the accuracy curves, so a signal that detects it would be a real early warning.)

Likely viva question: "Nanda says the transition can't be predicted ahead of time. Why did you expect it could?"

Module 2

Your training setup

This module describes the experiment that every predictor shares: the task, the data split, the model and its loss, the optimiser, the time axis, and the seeds. None of it depends on which predictor you're studying. Read it in order, because each section uses a number worked out in the one before: 12,769 pairs (Section 2.1) are split into 3,830 and 8,939 (Section 2.2); the model turns one pair into 113 scores and a loss (Section 2.3); AdamW and weight decay change the weights once per epoch (Sections 2.4 and 2.5); and five seeds give five runs (Section 2.6).

2.1 Modular addition task

In plain words: Every run in your benchmark learns the same small task: add two numbers and keep only the remainder after dividing by 113. For example, $100 + 50 = 150$, and the remainder of 150 after dividing by 113 is 37. The task is tiny and completely known: you can list every possible question and its answer. That's what makes it useful. Because the full list is known, you can choose exactly which pairs the model sees in training and which are kept back for testing (Section 2.2), and you can say exactly what "learning the rule" means. This section counts the pairs and works out one answer, so that every later number has a source.

Definition: The model learns $c = (a + b) \bmod p$ **(N3)** for all inputs $a, b \in \{0, \ldots, p-1\}$, with $p = 113$ **(N1)**.

Why it matters and how to read it: It's the standard grokking task of Power et al. [1] and Nanda et al. [2], so your results can be compared with theirs.

Two numbers come from this section and are used everywhere after it. The first is $N = 12{,}769$, the number of all pairs: the pool that the training and test sets are cut from. The second is $p = 113$, the number of possible answers. Because c is a remainder after dividing by p, it always lies in $\{0, \ldots, 112\}$, so the model must pick one of 113 answers and produces 113 scores per pair (Section 2.3). So p plays two roles: it's the number of values each input can take, and it's the number of possible answers.

Worked example: The derivation counts the pairs and works out one answer.

Derivation (H.K. Dass layout): number of pairs and one answer

Given:

1. $p = 113$ (**N1**, `nanda_unified.yaml`).
2. Each of a and b takes p values, 0 to $p - 1$ (**N2**).
3. Example inputs $a = 100$, $b = 50$.

Let:

1. p: the modulus. Integer, the same in every seed and epoch. Positive. Role: size of the task.

2. N: the number of all pairs. Integer, fixed before training. Positive. Role: the pool the split is cut from.
3. a, b: the inputs of one pair. Integers in $\{0, \ldots, 112\}$. Role: what the model reads.
4. c: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: its label.

To find: N and c for $a = 100$, $b = 50$.

Solution:

$$
\begin{aligned}
N &= p \times p && \because\ p \text{ choices of } a \text{ times } p \text{ choices of } b \text{ (Given 2)} \\
&= 113 \times 113 && \because\ \text{substitution of Given 1} \\
&= 12769 && \because\ \text{multiplication} \\
c &= (a + b) \bmod p && \because\ \text{definition of } c \textbf{ (N3)} \\
&= (100 + 50) \bmod 113 && \because\ \text{substitution of Given 1 and Given 3} \\
&= 150 \bmod 113 && \because\ \text{addition} \\
&= 150 - 1 \times 113 && \because\ 113 \text{ fits once into } 150 \textbf{ (A4)} \\
&= 37 && \because\ \text{subtraction}
\end{aligned}
$$

$\therefore$ There are 12,769 pairs, and the pair $(100, 50)$ has the answer 37.

$$\boxed{N = 12769, \quad c = 37}$$

Self-check: How many possible answers does the model choose between? ($p = 113$: the integers 0 to 112, one logit each, because $(a + b) \bmod 113$ is always a remainder from 0 to 112.)

Likely viva question: "Why $p = 113$ and not another prime?" (It matches Nanda et al. 2023 [2].)

2.2 Train/test split (30%)

In plain words: If you test a model on the same pairs it was trained on, you learn nothing about whether it has the rule: it could simply have stored those answers. So the 12,769 pairs are cut, once per seed, into two groups. The smaller group is used for training. The larger group is never used to change the weights; it's used only to check the model's answers. Everything about grokking is measured on that second group, so this section is where the meaning of G_i is protected: if any pair crossed from one group to the other, test accuracy would no longer measure generalisation.

Definition: A fraction $f = 0.3$ **(N6)** of the N pairs forms the training set D_{train}; the other 70% form the test set D_{test} **(N7)**, which is never used to change the weights.

Why it matters and how to read it: G_i is measured on D_{test}, so the split must be fixed per seed and no test pair may leak into training. In your code the 30% is hard-coded in `get_dataloaders`; the config's `train_fraction` doesn't change it. That's an audit finding.

Two things follow from this. First, the split is part of the run: it's drawn from the seed's random numbers **(N8)**, so the same seed always gives the same split. That's

what lets you rebuild it later to evaluate a saved checkpoint (Section 2.5). Second, the sizes are set by one number, the train fraction $f = 0.3$. Because that number is hard-coded, anyone who edits the config and expects a different split would silently get 30% anyway. Note also the sizes themselves: the test set is more than twice the size of the training set, so test accuracy is computed over 8,939 pairs.

Worked example: The derivation computes the two set sizes.

Derivation (H.K. Dass layout): sizes of the training and test sets

Given:

1. $N = 12769$ (Section 2.1).
2. $f = 0.3$, hard-coded as `train_size = int(0.3 * len(...))` (**N6**).
3. `int()` of a positive number equals its floor (**A3**).

Let:

1. N: the number of all pairs. Integer, fixed before training. Role: the pool being split.
2. f: the train fraction. Scalar in $(0, 1)$. Role: share of pairs used for training.
3. N_{train}: the size of D_{train}. Integer, fixed per seed before training. Role: pairs used to change the weights.
4. N_{test}: the size of D_{test}. Integer, fixed per seed before training. Role: pairs that measure generalisation.

To find: N_{train} and N_{test}.

Solution:

$$
\begin{aligned}
N_{\text{train}} &= \lfloor f \times N \rfloor && \because \texttt{int(0.3 * len(...))} \text{ and Given 3} \\
&= \lfloor 0.3 \times 12769 \rfloor && \because \text{substitution of Given 1 and Given 2} \\
&= \lfloor 3830.7 \rfloor && \because \text{multiplication} \\
&= 3830 && \because \text{floor drops the 0.7 (\textbf{A3})} \\
N_{\text{test}} &= N - N_{\text{train}} && \because \text{every pair not in } D_{\text{train}} \text{ is in } D_{\text{test}} \\
&= 12769 - 3830 && \because \text{substitution of Given 1 and the line above} \\
&= 8939 && \because \text{subtraction}
\end{aligned}
$$

$\therefore$ Each run trains on 3,830 pairs and tests on 8,939 pairs.

$$\boxed{N_{\text{train}} = 3830, \quad N_{\text{test}} = 8939}$$

Self-check: What goes wrong if a training pair appears in D_{test}? (The model can get that pair right by memorising it, so acc_{test} overstates generalisation. With enough leaked pairs acc_{test} could pass 0.9 too soon, and G_i would come too early.)

Likely viva question: "How do you guarantee the test set is truly unseen, and that it's the same split when you reload a checkpoint?" (The seed fixes the split; rebuilding it with the same seed gives the same D_{test}.)

2.3 Transformer, logits and cross-entropy loss

In plain words: This section follows one pair through the model. The model reads the two inputs and gives back 113 numbers, one score for each possible answer; the answer with the biggest score is its answer. To train it, you need a single number that says how good those scores are, and that number is the loss. The section goes in three steps: scores (the logits), then probabilities (the softmax), then the loss (cross-entropy). You don't need the inside of the model for any of this. What you do need is to know what comes out of it, because every predictor in the benchmark is computed either from the weights θ_t or from these outputs.

Definition: The model (**N5**) is a one-layer, four-head transformer. For one pair it reads (a, b) and the weights θ_t (**N10**) and returns the logits z (**N11**), a vector with 113 entries, one score per possible answer. The softmax turns them into probabilities $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**). The loss of the pair is the cross-entropy $\ell = -\ln q_c$ (**N14**), and the training loss $L(\theta_t)$ is the average of ℓ over D_{train} (**N15**).

Why it matters and how to read it: Every predictor is computed from this shared model's weights θ_t or its outputs.

Read the chain one link at a time. The logits z_k can be any real numbers, positive or negative, so they aren't probabilities. The softmax fixes that: each e^{z_k} is positive, and dividing by their sum makes the 113 values add up to 1. A bigger logit always gives a bigger probability, so the softmax never changes which answer the model picks ($\hat{k}$, **N12**); it only turns scores into shares. The loss then looks at just one of the 113 probabilities, q_c, the share given to the correct answer. If q_c is close to 1 the loss is close to 0; as q_c falls towards 0 the loss grows without limit. So the loss rewards putting a high probability on the right answer. The training loss $L(\theta_t)$ averages this over all 3,830 training pairs, and training is the process of making that average small. The two worked examples show how large the loss is at a poor and at a good answer, and that only the differences between logits matter, not their overall level.

Worked example: Two derivations: the loss at two probabilities, and why adding one constant to all logits leaves the loss unchanged.

Derivation (H.K. Dass layout): the loss at $q_c = 0.25$ and at $q_c = 0.99$

Given:

1. $\ell = -\ln q_c$ (**N14**).
2. Two example probabilities of the correct answer: 0.25 and 0.99.

Let:

1. q_c: the probability the model gives the correct answer c of one pair, at θ_t. Scalar in $(0, 1)$. Role: what the loss judges.
2. $\ell_{0.25}$: the loss of that pair when $q_c = 0.25$. Scalar ≥ 0. Role: loss of a poor answer.
3. $\ell_{0.99}$: the loss of that pair when $q_c = 0.99$. Scalar ≥ 0. Role: loss of a confident correct answer.

To find: $\ell_{0.25}$ and $\ell_{0.99}$.

Solution:

$$
\begin{aligned}
\ell_{0.25} &= -\ln 0.25 && \because \text{ Given 1 with } q_c = 0.25 \\
&= -(-1.3863\ldots) && \because \ln 0.25 = -1.3863\ldots \\
&= 1.3863\ldots && \because -(-x) = x \\
\ell_{0.99} &= -\ln 0.99 && \because \text{ Given 1 with } q_c = 0.99 \\
&= -(-0.01005\ldots) && \because \ln 0.99 = -0.01005\ldots \\
&= 0.01005\ldots && \because -(-x) = x
\end{aligned}
$$

$\therefore$ A probability of 0.25 on the correct answer costs about 1.39; a probability of 0.99 costs about 0.01. (Here "..." means the digits go on; the boxed values are rounded.)

$$\boxed{\ell_{0.25} = 1.39 \text{ (2 d.p.)}, \quad \ell_{0.99} = 0.01 \text{ (2 d.p.)}}$$

Derivation (H.K. Dass layout): adding the same constant to all logits doesn't change the loss

Given:

1. $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**).
2. $\ell = -\ln q_c$ (**N14**).
3. $e^{x+y} = e^x e^y$ for scalars x, y (rule of exponents).

Let:

1. z: the logits of one pair at θ_t. Vector with 113 entries $z_0, \ldots, z_{112}$. Role: the model's scores.
2. c: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: picks which probability the loss uses.
3. s: a constant. Scalar, any sign, the same for all 113 entries. Role: the shift being tested.
4. z': the shifted logits, $z'_k = z_k + s$ for every k. Vector with 113 entries. Role: the changed input.
5. q'_c: the softmax probability of c computed from z'. Scalar in $(0, 1)$.
6. ℓ': the loss computed from z', $\ell' = -\ln q'_c$. Scalar ≥ 0.

To find: Show that $\ell' = \ell$.

Solution:

$$
\begin{aligned}
q'_c &= \frac{e^{z'_c}}{\sum_j e^{z'_j}} && \because \text{ Given 1 applied to } z' \\
&= \frac{e^{z_c+s}}{\sum_j e^{z_j+s}} && \because \text{ substitution } z'_k = z_k + s \text{ (Let 4)} \\
&= \frac{e^s e^{z_c}}{\sum_j e^{z_j+s}} && \because \text{ Given 3 on the numerator} \\
&= \frac{e^s e^{z_c}}{\sum_j e^s e^{z_j}} && \because \text{ Given 3 on each term of the denominator}
\end{aligned}
$$

$$
\begin{aligned}
&= \frac{e^s e^{z_c}}{e^s \sum_j e^{z_j}} && \because\ e^s \text{ is the same scalar in every term, so it comes out of the sum} \\
&= \frac{e^{z_c}}{\sum_j e^{z_j}} && \because\ e^s/e^s = 1, \text{ since } e^s > 0 \\
&= q_c && \because\ \text{Given 1} \\
\ell' &= -\ln q'_c && \because\ \text{definition of } \ell' \text{ (Let 6)} \\
&= -\ln q_c && \because\ q'_c = q_c, \text{ shown above} \\
&= \ell && \because\ \text{Given 2}
\end{aligned}
$$

$\therefore$ Adding the same number to all 113 logits leaves every probability, and so the loss, unchanged.

$$\boxed{\ell' = \ell}$$

Reading the answer: The first result shows how sharply the loss separates answers: going from $q_c = 0.25$ to $q_c = 0.99$ cuts the loss from about 1.39 to about 0.01. The second shows that the scores only matter relative to each other: raising all 113 logits by the same amount changes no probability, so the loss can't see it.

Self-check: Why does adding the same constant to all 113 logits not change the loss? (The derivation above: the shift multiplies the top and every term of the bottom of the softmax by the same e^s, so it cancels. Every q_k, and so the loss, is unchanged.)

Likely viva question: "Is your model exactly Nanda's?" (No. It lacks the attention output matrix W_O (**N5**). That's a known gap.)

2.4 AdamW and weight decay

In plain words: Training changes the weights a little at every epoch. Two things decide each change. One is the gradient, which says in which direction the training loss grows; the optimiser, AdamW, turns the gradients into a step the other way, downhill. The other is weight decay, which shrinks every weight a little towards zero at every update, whatever the data says. This section writes the update as one equation and then rewrites it so that the shrink shows up as a single factor. It matters because the shrink happens in every run, at every epoch, and you have to be able to say whether a change in a signal comes from grokking or just from this steady shrinking.

Definition: AdamW (**N18**) is the optimiser that turns gradients into weight updates. Weight decay λ (**N19**) shrinks every weight a little at each update. The full update is the equation $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).

Why it matters and how to read it: You use $\lambda = 1.0$ with $\beta_1 = 0.9$, $\beta_2 = 0.98$, as in Nanda. Weight decay is believed to drive grokking. It also shrinks the weights on its own, in every run, so any signal computed from the size of the weights can move because of the optimiser rather than because grokking is coming. That's a confound (**N20**).

Read the update equation of **N20** term by term. θ_t is where the weights are now. $-\eta\, u_t$ is the AdamW step: u_t is built from the gradients so far and points roughly uphill, so subtracting it moves the weights downhill, towards lower training loss. $-\eta\lambda\, \theta_t$ is the

decay. It's proportional to the weights themselves, so a large weight is pulled in more than a small one, and every weight is pulled towards 0. "Decoupled" means this pull is applied directly to θ_t, separately from u_t, so its size doesn't depend on the gradient at all. The derivation below shows that, with your $\lambda = 1.0$, the decay is the same as multiplying every weight by one fixed number before the AdamW step.

Worked example: The derivation rewrites the update to show the decay as a shrink factor.

Derivation (H.K. Dass layout): the weight decay as a shrink factor

Given:

1. $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).
2. $\lambda = 1.0$ (**N19**, `nanda_unified.yaml`).

Let:

1. θ_t: the weights after epoch t. Vector with M entries. Role: what's being updated.
2. θ_{t+1}: the weights after epoch $t+1$. Vector with M entries.
3. u_t: the AdamW step at epoch t. Vector with M entries, roughly uphill, so $-\eta u_t$ is downhill.
4. η: the learning rate. Scalar > 0.
5. λ: the weight decay. Scalar ≥ 0.

To find: θ_{t+1} written as a scaled copy of θ_t plus the AdamW step, for $\lambda = 1.0$.

Solution:

$$
\begin{aligned}
\theta_{t+1} &= \theta_t - \eta\, u_t - \eta\lambda\, \theta_t && \because \text{ Given 1} \\
&= \theta_t - \eta\lambda\, \theta_t - \eta\, u_t && \because \text{ reordering the two subtracted terms} \\
&= (1 - \eta\lambda)\, \theta_t - \eta\, u_t && \because \text{ factoring the scalar out of } \theta_t - \eta\lambda\theta_t \\
&= (1 - \eta \times 1.0)\, \theta_t - \eta\, u_t && \because \text{ substitution of Given 2} \\
&= (1 - \eta)\, \theta_t - \eta\, u_t && \because \ \eta \times 1.0 = \eta
\end{aligned}
$$

$\therefore$ At every update the weights are first scaled by $(1 - \eta)$, which is below 1, and then moved by the AdamW step. The scaling pulls every weight towards 0 whatever the data does.

$$\boxed{\theta_{t+1} = (1 - \eta)\, \theta_t - \eta\, u_t}$$

Reading the answer: Each update now reads as two moves in a row: first multiply every weight by the same number $(1-\eta)$, which is below 1 because $\eta > 0$; then take the AdamW step. The first move doesn't look at the data at all. So if a signal computed from the size of the weights falls steadily, the factor $(1 - \eta)$ alone could explain it, and you need some other evidence before calling it a sign that grokking is coming.

Self-check: Why could a weight-based signal move because of the optimiser rather than because grokking is coming? (The factor $(1 - \eta)$ shrinks θ_t at every update, in every run.)

Likely viva question: "Did you vary λ? If not, how can you separate your signal from the effect of weight decay?"

2.5 Epochs and checkpoints

In plain words: Every time in the benchmark is measured in epochs. An epoch is one full pass over the training pairs, and because your runs use all 3,830 training pairs at once, an epoch is also exactly one weight update. So "epoch t" and "after the t-th update" mean the same thing. Alongside the time axis there's the question of what is saved. The weights are saved as checkpoints, but not at every epoch. This section explains how often, and why that sets a limit on how precisely anything computed from checkpoints can place an epoch.

Definition: An epoch (**N9**) is one full pass over D_{train}. Your runs are full batch, so one epoch is one weight update. A checkpoint (**N21**) is a saved copy of θ_t at one epoch.

Why it matters and how to read it: All runs last 40,000 epochs. G_i and every event epoch are reported in epochs, and several predictors are computed afterwards from checkpoints. Checkpoints are saved every 50 epochs and evaluated every 100, so a quantity computed from checkpoint evaluations can only place an epoch to within 100 epochs.

That limit is the time resolution. If something changes between two checkpoint evaluations, all you know from the evaluations is that it happened somewhere in that gap of 100 epochs. It matters because the lead margin in Section 4.2 is also 100 epochs, so the uncertainty from the evaluation spacing is as large as the margin itself. The derivation below shows the gap for seed 0's grok epoch.

Worked example: The derivation finds the two checkpoint evaluations around G_0, assuming evaluations fall on multiples of 100.

Derivation (H.K. Dass layout): checkpoint evaluations around G_0

Given:

1. $G_0 = 12987$ (**N26**).
2. Checkpoint evaluations every 100 epochs, taken here to be at multiples of 100 (**N21**).

Let:

1. G_0: the grok epoch of seed 0. Integer epoch.
2. t_-: the last evaluated epoch at or before G_0. Integer, a multiple of 100.
3. t_+: the first evaluated epoch after G_0. Integer, a multiple of 100.

To find: t_- and t_+.

Solution:

$$
\begin{aligned}
t_- &= 100 \times \left\lfloor \frac{G_0}{100} \right\rfloor && \because \text{ largest multiple of 100 that's} \leq G_0 \text{ (\textbf{A3})} \\
&= 100 \times \lfloor 129.87 \rfloor && \because \text{ substitution of Given 1, and } 12987/100 = 129.87 \\
&= 100 \times 129 && \because \text{ floor} \\
&= 12900 && \because \text{ multiplication}
\end{aligned}
$$

$$
\begin{aligned}
t_+ &= t_- + 100 && \because \text{ evaluations are 100 epochs apart (Given 2)} \\
&= 13000 && \because \text{ addition}
\end{aligned}
$$

$\therefore$ From checkpoint evaluations alone you'd only know that seed 0 groks between epochs 12,900 and 13,000.

$$\boxed{t_- = 12900, \quad t_+ = 13000}$$

Your G_i values (e.g. 12987) aren't multiples of 50 or 100, so they come from an accuracy curve recorded more often than the checkpoints. Check which curve `analyze_nanda_unified.py` reads.

Self-check: Why must the data split be rebuilt with the same seed when re-reading a checkpoint? (A checkpoint holds only the weights. To evaluate it on test pairs you must rebuild the same D_{test}; with a different split the rebuilt D_{test} would contain pairs the checkpoint was trained on, and acc_{test} would come out too high.)

Likely viva question: "What's your time resolution, and could it change any verdict?" (The lead margin of 100 epochs, Section 4.2, is the same size as the evaluation spacing.)

2.6 Seeds and why five

In plain words: One training run gives you one story: one grok epoch and, for a predictor, one event. You can't learn from one story whether a warning moves with grok or just happens to sit before it. So the benchmark repeats the run with different random starting points. Each repeat is fixed by a seed, a label that sets the random numbers, so the same seed always gives the same run. This section introduces the five seeds and computes the mean grok epoch, which gives you a feel for the time scale everything else is measured against.

Definition: A seed i (**N8**) fixes the random numbers of one run (the initial weights and the data split), so each seed is one independent, repeatable run.

Why it matters and how to read it: Your benchmark uses 5 seeds, $i \in \{0, 1, 2, 3, 4\}$. Different seeds grok at different epochs, and that spread is what lets you test whether a predictor tracks when grok happens, not just that it happens.

Think about what a seed changes: the initial weights θ_0 and which pairs land in D_{train}. Everything else, the task, the model, the optimiser and the 40,000 epochs, is the same in every run. So the differences between seeds come only from these random draws, and a good warning should respond to them in the same way grok does: if seed 4 groks late, its warning should come late too. The mean grok epoch below is a summary only; no PASS condition uses it. It tells you that a typical run groks around epoch 12,800, roughly a third of the way through the 40,000 epochs.

Worked example: The derivation computes the mean grok epoch.

Derivation (H.K. Dass layout): the mean grok epoch over the five seeds

Given:

1. $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N26)**.
2. The mean of 5 numbers is their sum divided by 5.

Let:

1. G_i: the grok epoch of seed i. Integer epoch, positive.
2. $\bar{G}$: the mean grok epoch. Scalar epoch, positive. Role: a typical grok time.

To find: $\bar{G}$.

Solution:

$$\begin{aligned}
\bar{G} &= \frac{G_0 + G_1 + G_2 + G_3 + G_4}{5} && \because \text{ Given 2} \\
&= \frac{12987 + 6921 + 9510 + 11019 + 23747}{5} && \because \text{ substitution of Given 1} \\
&= \frac{64184}{5} && \because \text{ addition} \\
&= 12836.8 && \because \text{ division}
\end{aligned}$$

$\therefore$ The five runs grok at epoch 12,836.8 on average, ranging from 6,921 (seed 1) to 23,747 (seed 4).

$$\boxed{\bar{G} = 12836.8}$$

Self-check: Why is one seed not enough to judge a predictor? (One run gives one grok epoch and one event epoch. One point can't show whether the event moves with grok; you need several runs that grok at different epochs.)

Likely viva question: "Five is very few. What can and can't you conclude from five seeds?" (See Module 5.)

Module 3

Measuring the outcome

Module 2 set up the experiment; this module says how its outcome is measured. There's exactly one outcome per run, the grok epoch G_i, and it's computed from test accuracy alone. No predictor is involved in computing it, so it's the fixed target that every predictor is compared with. Section 3.1 explains test accuracy and what the 0.9 threshold means as a count of pairs; Section 3.2 defines G_i and measures how far apart your five values are.

3.1 Test accuracy

In plain words: Test accuracy is the simplest number in the book: out of the 8,939 test pairs, what fraction does the model answer correctly? It's also the most important one, because grokking is defined from it and from nothing else. This section turns the 0.9 threshold into a plain count of pairs, so that when you read "test accuracy above 0.9" you can picture what it means: at least 8,046 test pairs right.

Definition: Test accuracy $\text{acc}_{\text{test}}(t)$ (**N22**) is the fraction of the 8,939 test pairs on which the predicted answer $\hat{k}$ (**N12**) equals the correct answer c, using θ_t.

Why it matters and how to read it: It's the only thing that defines grok in your benchmark.

Accuracy only asks whether the model's chosen answer $\hat{k}$ equals the correct c; it ignores how confident the model is. That's why it's used rather than test loss. The loss changes whenever the probabilities change, even on pairs whose answer is already right, so it can drift without a single answer changing. Accuracy moves only when an answer changes from wrong to right or back, which is exactly what generalisation is about. Each test pair is $1/8939$ of the total, about 0.0001, so one more pair right moves test accuracy by about one ten-thousandth.

Worked example: The derivation finds how many test pairs must be right for acc_{test} to pass the grok threshold 0.9.

Derivation (H.K. Dass layout): correct test pairs needed for $\text{acc}_{\text{test}} > 0.9$

Given:

1. $N_{\text{test}} = 8939$ (Section 2.2).
2. $\text{acc}_{\text{test}}(t) = (\text{number of correct test pairs})/N_{\text{test}}$ (**N22**).
3. Grok needs $\text{acc}_{\text{test}}(t) > 0.9$ (**N26**).

Let:

1. K: the number of test pairs with $\hat{k} = c$ at θ_t. Integer from 0 to 8939. Role: the count behind acc_{test}.
2. $K_{\min}$: the smallest K with $K/N_{\text{test}} > 0.9$. Integer.

To find: K_{min}.

Solution:

$$\frac{K}{N_{\text{test}}} > 0.9 \qquad \because \text{ Given 2 put into Given 3}$$

$$K > 0.9 \times N_{\text{test}} \qquad \because \text{ multiplying both sides by } N_{\text{test}} > 0$$

$$K > 0.9 \times 8939 \qquad \because \text{ substitution of Given 1}$$

$$K > 8045.1 \qquad \because \text{ multiplication}$$

$$K_{\text{min}} = 8046 \qquad \because \text{ } K \text{ is an integer; 8046 is the first integer above 8045.1}$$

$\therefore$ At least 8,046 of the 8,939 test pairs must be right. For example, 8,100 right gives $8100/8939 = 0.906$ (3 d.p.), which passes.

$$\boxed{K_{\text{min}} = 8046}$$

Self-check: Why not use test loss instead? (Loss also changes when the model just gets more or less confident on pairs it already gets right; accuracy only changes when answers change, and a change of answers on unseen pairs is what generalisation means.)

Likely viva question: "Why accuracy and not loss to define grokking?"

3.2 The grok epoch G_i

In plain words: The test-accuracy curve of one run is thousands of numbers. To compare a warning with grok, you need grok as a single epoch. The grok epoch G_i is that epoch: the first time test accuracy goes above 0.9 in the run of seed i. This section gives the five values and measures how far apart they are. That spread is what makes the rest of the benchmark possible: if all five runs grokked at the same epoch, there'd be no way to tell a warning that follows grok from one that just fires at a fixed time.

Definition: The grok epoch of seed i is $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ (**N26**): the first epoch at which test accuracy goes above 0.9.

Why it matters and how to read it: It's the target every predictor is compared with, and it's the same for all predictors. Recomputing it from the saved curves reproduces your saved values.

Two details of the definition are worth reading slowly. "First epoch" means later dips don't matter: once acc_{test} has gone above 0.9, G_i is fixed, even if accuracy wobbles afterwards. And "above 0.9" is strict: an epoch with acc_{test} exactly 0.9 doesn't count. The threshold 0.9 is fixed in the config before any predictor is scored, so it can't be moved to suit one predictor, and the G_i come from training alone, so they're the same whichever predictor you're judging.

Worked example: $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. The derivation measures their spread.

Derivation (H.K. Dass layout): the spread of the grok epochs

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N26)**.

Let:

1. $G_{\max}$: the largest G_i. Integer epoch.
2. $G_{\min}$: the smallest G_i. Integer epoch.
3. Δ: the range, $G_{\max} - G_{\min}$. Integer number of epochs, ≥ 0. Role: how far apart the grok times are.

To find: Δ.

Solution:

$$
\begin{aligned}
G_{\max} &= 23747 && \because \text{ seed 4 has the largest value in Given 1} \\
G_{\min} &= 6921 && \because \text{ seed 1 has the smallest value in Given 1} \\
\Delta &= G_{\max} - G_{\min} && \because \text{ definition of } \Delta \text{ (Let 3)} \\
&= 23747 - 6921 && \because \text{ substitution of the two lines above} \\
&= 16826 && \because \text{ subtraction}
\end{aligned}
$$

$\therefore$ The grok epochs spread over 16,826 epochs. That spread is what a predictor's events must follow (Module 5).

$$\boxed{\Delta = 16826}$$

Reading the answer: The latest run groks more than three times later than the earliest ($23747/6921$ is about 3.4). So the five runs really do differ in when they grok, and a warning that tracks grok has plenty of room to show it.

Self-check: Would G_i change much with a 0.95 threshold? Why does it matter that the threshold is fixed? (It moves G_i later by however long acc_{test} takes to go from 0.9 to 0.95. Fixing it stops it being picked after the fact to suit a predictor.)

Likely viva question: "Why 0.9? Is your result sensitive to that choice?"

Module 4

What a predictor is

From here on, the book is about warnings. This module says what a predictor is, what it outputs (one event epoch per seed, or a miss), and the two numbers that say whether those events come early enough: the lead fraction (Sections 4.2 and 4.3) and the mean relative lead (Section 4.4). Section 4.5 then shows how misses are handled. Every example uses List A, a made-up event list, so nothing in this module is about any one of your predictors.

4.1 Predictor and event epoch P_i

In plain words: Everything so far was about the runs and their outcome. Now comes the thing being tested. A predictor is a warning system with two parts: something you measure during training, called the signal, and a fixed rule that looks at the signal's curve and names one epoch. That named epoch is the event, the predictor's warning for that run. If the rule never names an epoch, that run is a miss. This section fixes the vocabulary and introduces List A, a made-up set of event epochs that the next modules use to show every calculation without referring to any real predictor of yours.

Definition: A predictor (**N31**) is a (signal, rule) pair: a signal y_t (**N27**) tracked during training, plus a rule (**N29**) that names one epoch P_i, the event epoch, as the warning in the run of seed i. If the rule never names an epoch, that's a miss, stored as `None`.

Why it matters and how to read it: Each of your eight predictors is a (signal, rule) pair. If the rule or one of its settings (**N30**) changes, it's a different predictor, so every verdict belongs to one exact rule.

This is stricter than it sounds. The same signal with a different window, or with a different number of skipped epochs, is a new predictor and needs its own verdict. The reason is that the verdict is about the events, and the events come from the rule: change the rule and you change the events. For scoring, a predictor's whole output is therefore five numbers, one P_i per seed, some of which may be `None`. All the scoring in Modules 4 to 6 works on those five numbers and the five G_i; nothing else about the signal is used.

Worked example: Suppose some rule fires at the epochs below. This is **List A**, an illustrative event list (**N32**); it belongs to no predictor of yours. Modules 4 to 6 use it to show every calculation.

$$\text{List A:} \quad (P_0, P_1, P_2, P_3, P_4) = (3500,\ 6850,\ 2900,\ 4100,\ 7800).$$

All five seeds fire, so there's no `None`. There's no calculation here. Look at List A next to the grok epochs. Seed 1's event (6850) is very close to its grok epoch (6921), and the order of the events doesn't match the order of the grok epochs. Both features matter later (Sections 4.2 and 5.1).

Self-check: What should the code do if the rule never fires? (Store `None`; the seed is then dropped from the firing set F, **N33**.)

Likely viva question: "Which exact rule is the official one for this predictor?" (Each predictor needs one named rule, with its settings, fixed before scoring. Two candidate rules are two predictors.)

4.2 Lead and the 100-epoch margin

In plain words: A warning is only useful if it comes early enough to be a warning. An event at epoch 6,900 in a run that groks at 6,921 is before grok, but only by 21 epochs, less than the 100-epoch gap between two checkpoint evaluations. The benchmark therefore asks for more than "before": the event must come more than 100 epochs before grok. A seed whose event clears that bar is said to lead. This section applies the test to two seeds of List A, one that clears it easily and one that's before grok but too close.

Definition: A firing seed i leads (**N34**) only if its event comes more than 100 epochs before grok: $P_i < G_i - 100$.

Why it matters and how to read it: The margin stops an event that fires at grok, or just before it, from counting as a warning.

To read the condition $P_i < G_i - 100$, first compute the cutoff $G_i - 100$ for that seed, then ask whether the event is below it. The cutoff is different for every seed, because each seed has its own grok epoch. Note what the test doesn't care about: an event at epoch 3,500 and one at epoch 50 both lead in seed 0. How early the event is gets measured separately, by the relative lead (Section 4.4). Note also the point from Section 2.5: the margin is the same size as the evaluation spacing, so for a quantity computed from checkpoint evaluations, an event close to the cutoff can lead or not depending on where the evaluations fall.

Worked example: The derivation checks seeds 0 and 1 of List A (Section 4.1). One leads; the other fires before grok but inside the margin.

Derivation (H.K. Dass layout): do seeds 0 and 1 of List A lead?

Given:

1. $G_0 = 12987$ and $G_1 = 6921$ (**N26**).
2. $P_0 = 3500$ and $P_1 = 6850$ (List A, Section 4.1).
3. Seed i leads if $P_i < G_i - 100$ (**N34**).

Let:

1. G_i: the grok epoch of seed i. Integer epoch.
2. P_i: the event epoch of seed i in List A. Scalar epoch.
3. T_i: the latest allowed event epoch for seed i, $T_i = G_i - 100$. Scalar epoch. Role: the cutoff an event must be below.

To find: Whether $P_0 < T_0$ and whether $P_1 < T_1$.

Solution:

$$
\begin{aligned}
T_0 &= G_0 - 100 && \because \text{ definition of } T_i \text{ (Let 3) with } i = 0 \\
&= 12987 - 100 && \because \text{ substitution of Given 1} \\
&= 12887 && \because \text{ subtraction} \\
P_0 = 3500 &< 12887 = T_0 && \because \text{ Given 2 and the line above: seed 0 leads} \\
T_1 &= G_1 - 100 && \because \text{ definition of } T_i \text{ (Let 3) with } i = 1 \\
&= 6921 - 100 && \because \text{ substitution of Given 1} \\
&= 6821 && \because \text{ subtraction} \\
P_1 = 6850 &> 6821 = T_1 && \because \text{ Given 2 and the line above: seed 1 doesn't lead}
\end{aligned}
$$

$\therefore$ Seed 0 leads. Seed 1 fires before grok ($6850 < 6921$), but only 71 epochs before, inside the 100-epoch margin, so it doesn't lead.

$$\boxed{P_0 < T_0 : \text{ seed 0 leads}; \quad P_1 > T_1 : \text{ seed 1 doesn't lead}}$$

Reading the answer: Seed 1 is the case to remember. It's before grok, so a quick look would call it early, but it misses the cutoff by 29 epochs ($6850 - 6821 = 29$).

Self-check: Does the `seeds_before_grok` count use the same margin? (No. It counts $G_i - P_i > 0$ (**N35**), so seed 1 of List A would count as early there but doesn't lead under the protocol. That's an audit finding.)

Likely viva question: "Why 100 epochs?"

4.3 Lead fraction

In plain words: Section 4.2 judged one seed at a time. A predictor needs one score across all its seeds, and the lead fraction is that score: of the seeds where the rule fired, what share led? It's the first of the three PASS conditions. This section works it out for List A, which turns out to sit exactly on the threshold.

Definition: The lead fraction (**N36**) is the share of firing seeds that lead: lead_frac = $\#\{i \in F : P_i < G_i - 100\}/n$, where F is the firing set and $n = \#F$ (**N33**).

Why it matters and how to read it: PASS needs lead_frac ≥ 0.8. With $n = 5$ that's at least 4 of 5 seeds; with $n = 4$ it's 4 of 4.

Notice the bottom of the fraction: it's n, the number of seeds that fired, not always 5. That choice matters when there are misses, and Section 4.5 looks at it closely. With five firing seeds the lead fraction can only be 0, 0.2, 0.4, 0.6, 0.8 or 1, so "at least 0.8" means "at most one seed fails to lead". The threshold allows one bad seed, but not two.

Worked example: The derivation computes lead_frac for List A, where all 5 seeds fire.

Derivation (H.K. Dass layout): lead_frac **of List A**

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N26)**.
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. All 5 seeds fire, so $F = \{0, 1, 2, 3, 4\}$ (Given 2 has no `None`).
4. $(G_0 - 100, \ldots, G_4 - 100) = (12887, 6821, 9410, 10919, 23647)$ (subtracting 100 from each entry of Given 1).
5. lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$ **(N36)**.

Let:

1. G_i, P_i: grok epoch and event epoch of seed i. Scalars.
2. n: the number of firing seeds. Integer.
3. $\mathcal{L}$: the set of seeds in F that lead. Set of seed labels. Role: the count on top of lead_frac.

To find: lead_frac.

Solution:

$$
\begin{aligned}
n &= \#F && \because \text{ definition of } n \text{ (\textbf{N33})} \\
&= 5 && \because \text{ Given 3} \\
P_0 = 3500 &< 12887 = G_0 - 100 && \because \text{ Given 2 and 4: seed 0 leads} \\
P_1 = 6850 &> 6821 = G_1 - 100 && \because \text{ Given 2 and 4: seed 1 doesn't lead (Section 4.2)} \\
P_2 = 2900 &< 9410 = G_2 - 100 && \because \text{ Given 2 and 4: seed 2 leads} \\
P_3 = 4100 &< 10919 = G_3 - 100 && \because \text{ Given 2 and 4: seed 3 leads} \\
P_4 = 7800 &< 23647 = G_4 - 100 && \because \text{ Given 2 and 4: seed 4 leads} \\
\mathcal{L} &= \{0, 2, 3, 4\} && \because \text{ the four seeds that lead, from the lines above} \\
\#\mathcal{L} &= 4 && \because \text{ counting (\textbf{A1})} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 5} \\
&= \frac{4}{5} && \because \text{ substitution of the lines above} \\
&= 0.8 && \because \text{ division}
\end{aligned}
$$

$\therefore$ Four of the five seeds lead, so List A sits exactly on the lead condition lead_frac ≥ 0.8 and meets it.

$$\boxed{\text{lead_frac} = 0.8}$$

Reading the answer: List A meets the condition with no room to spare. If one more seed had failed to lead, its lead fraction would drop to $3/5 = 0.6$ and the condition would fail.

Self-check: If one seed misses and one fires after grok, what is lead_frac? (The miss is dropped, so $n = 4$, and 3 of those 4 lead: lead_frac $= 3/4 = 0.75$, a fail.)

Likely viva question: "Is 4 out of 5 strong evidence?"

4.4 Relative lead and mean relative lead

In plain words: The lead test of Section 4.2 is yes-or-no: it doesn't say how early the event is. The relative lead adds that. It measures the warning time $G_i - P_i$ as a share of the grok epoch, so that seeds with very different grok epochs can be compared on one scale. A relative lead of 0.5 means the event came when half the time to grok had passed; 0.9 means it came when only a tenth had passed. This section computes it for one seed and then averages it over List A.

Definition: The relative lead of seed i (**N37**) is $r_i = (G_i - P_i)/G_i$: how early the event is, as a fraction of the grok epoch. The mean relative lead is $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$.

Why it matters and how to read it: PASS needs $0.05 \le \bar{r} \le 0.9$. Below 0.05 the event is too late to be useful. Above 0.9 it's so early that it's probably unrelated to grok.

The upper limit is the less obvious one. An event at epoch 100 in every run would lead in every seed and give relative leads close to 1, which looks like a superb warning. But an event that fires that early in every run, thousands of epochs before any grok, is unlikely to be reacting to this run's grok; the upper limit stops such events scoring well. Two reading tips. First, $r_i = 1 - P_i/G_i$ (shown in the second derivation), so r_i depends only on the ratio of event to grok. Second, $\bar{r}$ is an average, so one late seed can be hidden by four early ones; List A's seed 1 is an example.

Worked example: Two derivations: r_0 for List A, then $\bar{r}$ over all five seeds of List A.

Derivation (H.K. Dass layout): relative lead of seed 0, List A

Given:

1. $G_0 = 12987$ (**N26**).
2. $P_0 = 3500$ (List A, Section 4.1).
3. $r_i = (G_i - P_i)/G_i$ (**N37**).

Let:

1. G_0: grok epoch of seed 0. Integer epoch, positive.
2. P_0: event epoch of seed 0 in List A. Scalar epoch, positive.
3. r_0: relative lead of seed 0. Scalar; positive because $P_0 < G_0$.

To find: r_0.

Solution:

$$
\begin{aligned}
r_0 &= \frac{G_0 - P_0}{G_0} && \because \text{ Given 3 with } i = 0 \\
&= \frac{12987 - 3500}{12987} && \because \text{ substitution of Given 1 and Given 2} \\
&= \frac{9487}{12987} && \because \text{ subtraction} \\
&\approx 0.7305 && \because \text{ division, rounded to 4 d.p.}
\end{aligned}
$$

$\therefore$ Seed 0's event comes when only about 27% of the time to grok has passed, leaving about 73% of it as warning.

$$\boxed{r_0 = 0.7305 \text{ (4 d.p.)}}$$

Derivation (H.K. Dass layout): mean relative lead of List A

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. $n = 5$ and $F = \{0, 1, 2, 3, 4\}$ (Section 4.3).
4. $\bar{r} = \frac{1}{n} \sum_{i \in F} r_i$ with $r_i = (G_i - P_i)/G_i$ (**N37**).

Let:

1. r_i: relative lead of seed i. Scalar.
2. $\bar{r}$: mean relative lead. Scalar. Role: the third PASS condition.
3. S: the sum $\sum_{i=0}^{4} P_i/G_i$. Scalar, positive. Role: the part of $\bar{r}$ that depends on the events.

To find: $\bar{r}$.

Solution:

$$r_i = \frac{G_i - P_i}{G_i} \qquad \because \text{ Given 4}$$

$$= \frac{G_i}{G_i} - \frac{P_i}{G_i} \qquad \because \text{ splitting the fraction}$$

$$= 1 - \frac{P_i}{G_i} \qquad \because \ G_i/G_i = 1$$

$$\bar{r} = \frac{1}{5} \sum_{i=0}^{4} \left(1 - \frac{P_i}{G_i}\right) \qquad \because \text{ Given 4 with Given 3, and the line above}$$

$$= \frac{1}{5} \left(5 - \sum_{i=0}^{4} \frac{P_i}{G_i}\right) \qquad \because \text{ the five 1s add up to 5}$$

$$= 1 - \frac{1}{5} \sum_{i=0}^{4} \frac{P_i}{G_i} \qquad \because \text{ multiplying out } \tfrac{1}{5}$$

$$= 1 - \frac{S}{5} \qquad \because \text{ definition of } S \text{ (Let 3)}$$

$$S = \tfrac{3500}{12987} + \tfrac{6850}{6921} + \tfrac{2900}{9510} + \tfrac{4100}{11019} + \tfrac{7800}{23747} \qquad \because \text{ Let 3 with Given 1 and 2}$$

$$\approx 0.26950 + 0.98974 + 0.30494 + 0.37208 + 0.32846 \qquad \because \text{ each fraction rounded to 5 d.p.; later digits dropped}$$

$$= 2.26472 \qquad \because \text{ addition}$$

$$\bar{r} = 1 - \frac{2.26472}{5} \qquad \because \text{ substitution of } S \text{ into the line } \bar{r} = 1 - S/5$$

$$= 1 - 0.452944 \qquad \because \text{ division by 5}$$

$$= 0.547056 \qquad \because \text{ subtraction}$$

$\therefore$ The mean relative lead of List A is 0.547 to 3 d.p., inside $[0.05, 0.9]$, so List A meets this condition. Seed 1 contributes only $r_1 = 1 - 0.98974 = 0.01026$, because its event is just before grok.

$$\boxed{\bar{r} = 0.547 \text{ (3 d.p.)}, \quad 0.05 \leq \bar{r} \leq 0.9}$$

Reading the answer: Four of List A's seeds have relative leads between about 0.63 and 0.73; seed 1 has about 0.01. The average, 0.547, is comfortably inside the allowed range, even though one seed gives almost no warning at all.

Self-check: What $\bar{r}$ does an event at exactly half the grok epoch give? ($P_i = 0.5\,G_i$ gives $r_i = 0.5$ in every seed, so $\bar{r} = 0.5$.)

Likely viva question: "Why have an upper limit at all?" (Without it, an event at epoch 1 in every run would score $\bar{r} \approx 1$ and look like the best possible warning.)

4.5 Misses and `None`

In plain words: Sometimes a rule never fires in a run: the signal never does the thing the rule looks for. That's a miss, and the code stores `None`. The question is what a miss should count as. One answer is "no warning, so no lead". Your protocol gives a different answer: a miss is left out, and the scores are computed on the seeds that did fire. This section shows what that choice does to the lead fraction, because an examiner can ask about it for any predictor.

Definition: A miss (**N29**) is a seed where the rule never fires; the code stores `None`. Seeds with `None` are dropped before lead_frac, ρ and $\bar{r}$ are computed (**N33**). If fewer than 4 seeds fire ($n < 4$), the predictor fails automatically.

Why it matters and how to read it: How misses are handled changes lead_frac. Because a miss is dropped rather than counted as "no lead", four leads and one miss give lead_frac $= 4/4$, not $4/5$. That's a possible attack point for every predictor.

The protection against dropping too many seeds is the automatic fail for $n < 4$: a predictor that fires in fewer than four seeds fails whatever its other numbers are. So the protocol tolerates at most one miss. Keep two cases apart when you explain this. A miss has no event; it's dropped. A late event is an event that fails the margin; it's kept and counts as not leading. A late event lowers the lead fraction; a miss doesn't.

Worked example: The derivation works through one miss and four leads.

Derivation (H.K. Dass layout): one miss and four leads

Given:

1. One seed returns `None`; the other four fire and all four lead (illustrative case).
2. Seeds with `None` are dropped; $n < 4$ is an automatic fail (**N33**).
3. lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$ (**N36**).

Let:

1. F: the firing set. Set of seed labels.
2. n: the number of firing seeds, $n = \#F$. Integer.
3. $\mathcal{L}$: the seeds in F that lead. Set of seed labels.

To find: Whether the predictor fails automatically, and its lead_frac.

Solution:

$$
\begin{aligned}
n &= 5 - 1 && \because \text{ one of the 5 seeds is dropped (Given 1, 2)} \\
&= 4 && \because \text{ subtraction} \\
n &\geq 4 && \because \text{ so the automatic fail of Given 2 doesn't apply} \\
\#\mathcal{L} &= 4 && \because \text{ all four firing seeds lead (Given 1)} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 3} \\
&= \frac{4}{4} && \because \text{ substitution} \\
&= 1.0 && \because \text{ division}
\end{aligned}
$$

$\therefore$ The predictor isn't failed automatically, and its lead fraction is 1.0, the same as if all five seeds had led.

$$\boxed{n = 4, \quad \text{lead_frac} = 1.0}$$

Reading the answer: Under the other convention, counting the miss as "no lead", the same seeds would give $4/5 = 0.8$. Here both conventions meet the threshold, but they give different numbers, and with one miss and one late seed they'd give $3/4 = 0.75$ and $3/5 = 0.6$.

Self-check: Can you compute ρ with 4 seeds? (Yes: ρ is then computed on the 4 firing seeds, with $n = 4$ in **N40**; the missed seed takes no part in the ranking.)

Likely viva question: "How do you treat a seed where the signal never appears?"

Module 5

Spearman's ρ and small samples

Being early isn't enough. A useful warning must also move with grok: in runs that grok later, it should fire later. This module measures that with Spearman's ρ (Section 5.1), explains why ranks are used rather than raw epochs (Section 5.2), and then shows two consequences of having only five seeds: ρ can take only 21 values (Section 5.3), and a high ρ is not rare by luck alone (Section 5.4).

5.1 Spearman rank correlation ρ

In plain words: Suppose a rule fired at epoch 2,000 in every run. It would lead in every seed, but it would be a fixed clock: it would fire at 2,000 whether the run grokked at 7,000 or at 24,000, so it tells you nothing about when grok comes. To catch that, the benchmark asks a second question: do the events come in the same order as the grok epochs? If seed 1 groks first, does its event come first; if seed 4 groks last, does its event come last? Spearman's ρ answers that with one number. This section explains how it's computed from ranks and works it out for List A.

Definition: Spearman's ρ (**N40**) is a number from -1 to 1 that says whether two lists go up together in order, using only their ranks (**N38**). With no ties, $\rho = 1 - 6\sum_{i\in F} d_i^2/\big(n(n^2-1)\big)$, where $d_i = R(G_i) - R(P_i)$ is the rank difference of seed i.

Why it matters and how to read it: PASS needs $\rho \geq 0.9$: seeds that grok later must also have later events. That shows the event tracks this run's grok, not a fixed clock (an event at about the same epoch in every run, whatever G_i is).

To compute ρ you set the epoch values aside and keep only their order. Each list, the five G_i and the five P_i, is sorted, and each seed gets its position, its rank, from 1 (earliest) to 5 (latest). If the two lists of ranks are identical, every d_i is 0 and $\rho = 1$. The more the ranks disagree, the bigger the squared differences d_i^2 and the lower ρ goes; fully reversed order gives $\rho = -1$. Squaring makes a big disagreement count much more than small ones: one seed three places out costs 9, while one seed a single place out costs only 1.

Worked example: The derivation computes ρ for List A (Section 4.1). Its five P_i are all different, so there are no ties.

Derivation (H.K. Dass layout): ρ of List A

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N26**).
2. $(P_0, \ldots, P_4) = (3500, 6850, 2900, 4100, 7800)$ (List A, Section 4.1).
3. $n = 5$ (Section 4.3).
4. Rank: position from smallest, starting at 1 (**N38**).
5. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$ with $d_i = R(G_i) - R(P_i)$, when there are no ties (**N40**).

Let:

1. $R(G_i)$: rank of G_i among the five grok epochs. Integer from 1 to 5.
2. $R(P_i)$: rank of P_i among the five event epochs. Integer from 1 to 5.
3. d_i: rank difference of seed i. Integer; positive when seed i ranks higher in G than in P.
4. ρ: Spearman's rho of List A. Scalar in $[-1, 1]$.

To find: ρ.

Solution:

$$\begin{aligned}
\text{sorted } G &: 6921 < 9510 < 11019 < 12987 < 23747 && \because \text{ sorting Given 1: seeds 1, 2, 3, 0, 4}\\
(R(G_0), \ldots, R(G_4)) &= (4, 1, 2, 3, 5) && \because \text{ position of each seed in the line above}\\
\text{sorted } P &: 2900 < 3500 < 4100 < 6850 < 7800 && \because \text{ sorting Given 2: seeds 2, 0, 3, 1, 4}\\
(R(P_0), \ldots, R(P_4)) &= (2, 4, 1, 3, 5) && \because \text{ position of each seed in the line above}\\
(d_0, \ldots, d_4) &= (4-2,\ 1-4,\ 2-1,\ 3-3,\ 5-5) && \because\ d_i = R(G_i) - R(P_i) \text{ (Given 5)}\\
&= (2, -3, 1, 0, 0) && \because \text{ subtraction}\\
\sum d_i^2 &= 2^2 + (-3)^2 + 1^2 + 0^2 + 0^2 && \because \text{ squaring each } d_i\\
&= 4 + 9 + 1 + 0 + 0 && \because\ (-3)^2 = 9\\
&= 14 && \because \text{ addition}\\
\rho &= 1 - \frac{6\sum d_i^2}{n(n^2-1)} && \because \text{ Given 5}\\
&= 1 - \frac{6 \times 14}{5(5^2-1)} && \because \text{ substitution of } \textstyle\sum d_i^2 = 14 \text{ and Given 3}\\
&= 1 - \frac{6 \times 14}{5 \times 24} && \because\ 5^2 - 1 = 25 - 1 = 24\\
&= 1 - \frac{84}{5 \times 24} && \because\ 6 \times 14 = 84\\
&= 1 - \frac{84}{120} && \because\ 5 \times 24 = 120\\
&= 1 - 0.7 && \because\ 84/120 = 0.7\\
&= 0.3 && \because \text{ subtraction}
\end{aligned}$$

$\therefore$ List A ranks the seeds with $\rho = 0.3$, far below 0.9, so it fails the ρ condition. Its events don't follow the order of the grok epochs.

$$\boxed{\rho = 0.3 < 0.9}$$

Reading the answer: Most of List A's 14 comes from two seeds. Seed 1 groks first but its event is fourth ($d_1 = -3$, costing 9), and seed 0 groks fourth but its event is second ($d_0 = 2$, costing 4). One badly placed seed can pull ρ far below 0.9 on its own.

Self-check: What ρ does an event at a fixed epoch (the same in all seeds) get? (All P_i tie, so every $R(P_i)$ is 3, the $R(P)$ list is constant, and C in **N39** is undefined. Either way it can't reach 0.9, so it fails. Check whether your code returns NaN, "not a number", or 0 here.)

Likely viva question: "Why Spearman and not Pearson?"

5.2 Pearson vs Spearman

In plain words: There's a more familiar correlation, Pearson's, that works on the raw numbers rather than their ranks. It asks how close the points lie to a straight line. With many points that's often fine, but with five points one unusual value can decide the answer. Your grok epochs have exactly that feature: seed 4 groks at 23,747, nearly twice the next largest (12,987). This section compares the two correlations and shows how ρ is computed when two events tie, using List B.

Definition: Pearson's correlation $C(X, Y)$ (**N39**) measures how close the points (X_i, Y_i) are to a straight line, using the raw values. Spearman's ρ is Pearson's correlation of the ranks, $\rho = C\big(R(P), R(G)\big)$, so it uses only order. When there are ties, this is how ρ is computed.

Why it matters and how to read it: With 5 seeds, seed 4 ($G_4 = 23747$) is far from the rest and dominates Pearson's C. Spearman isn't pulled by one extreme value, because 23747 only counts as rank 5. Any event list that rises with G in exactly the same order has $\rho = 1$, even if the relationship is curved.

Spearman is Pearson applied to ranks. Replacing values by ranks keeps the order and throws away the distances, so 23,747 becomes simply "5", the same rank it would get if it were 13,000. That's also why Spearman doesn't care about the shape of the relationship: a straight line, a curve or a step all give $\rho = 1$, as long as the order matches. Ties need a rule, because two equal values can't both be "third". The rule (**N38**) gives each the average of the positions they share, so two values in positions 3 and 4 both get 3.5. With ties the short formula with d_i^2 is no longer exact, so ρ is computed as Pearson's correlation of the ranks.

Worked example: Suppose a rule fires at the epochs of **List B**, an illustrative event list (**N32**) with a tie between seeds 0 and 3:

$$\text{List B:} \quad (P_0, P_1, P_2, P_3, P_4) = (4000,\ 2000,\ 3000,\ 4000,\ 9000).$$

The derivation computes its ρ with the tie.

Derivation (H.K. Dass layout): ρ of List B, with a tie

Given:

1. $(P_0, \ldots, P_4) = (4000, 2000, 3000, 4000, 9000)$ (List B, above).
2. $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (Section 5.1).
3. Tied values get the average of the positions they occupy (**N38**).
4. $\rho = C\big(R(P), R(G)\big)$, with C from **N39**; $n = 5$.

Let:

1. $X_i = R(P_i)$: rank of the event of seed i. Scalar (can be a half).
2. $Y_i = R(G_i)$: rank of the grok epoch of seed i. Integer.
3. $\bar{X}$, $\bar{Y}$: the means of the X_i and of the Y_i. Scalars.
4. ρ: Spearman's rho with the tie. Scalar in $[-1, 1]$.

To find: ρ.

Solution:

$$\text{sorted } P : 2000 < 3000 < 4000 = 4000 < 9000 \quad \because \text{ sorting Given 1: seeds 1, 2, then 0 and 3 tied, then 4}$$

$$(X_0, \ldots, X_4) = (3.5, 1, 2, 3.5, 5) \quad \because \text{ seeds 0 and 3 tie for positions 3 and 4 (Given 3)}$$

$$\bar{X} = \frac{3.5 + 1 + 2 + 3.5 + 5}{5} \quad \because \text{ definition of the mean (\textbf{N39})}$$

$$= \frac{15}{5} \quad \because \text{ addition}$$

$$= 3 \quad \because \text{ division}$$

$$\bar{Y} = \frac{4 + 1 + 2 + 3 + 5}{5} \quad \because \text{ Given 2 and the definition of the mean}$$

$$= 3 \quad \because \ 15/5 = 3$$

$$(X_i - \bar{X}) = (0.5, -2, -1, 0.5, 2) \quad \because \text{ subtracting } \bar{X} = 3$$

$$(Y_i - \bar{Y}) = (1, -2, -1, 0, 2) \quad \because \text{ subtracting } \bar{Y} = 3$$

$$\sum (X_i - \bar{X})(Y_i - \bar{Y}) = 0.5 + 4 + 1 + 0 + 4 \quad \because \text{ multiplying the two lines above entry by entry}$$

$$= 9.5 \quad \because \text{ addition}$$

$$\sum (X_i - \bar{X})^2 = 0.25 + 4 + 1 + 0.25 + 4 \quad \because \text{ squaring each entry}$$

$$= 9.5 \quad \because \text{ addition}$$

$$\sum (Y_i - \bar{Y})^2 = 1 + 4 + 1 + 0 + 4 \quad \because \text{ squaring each entry}$$

$$= 10 \quad \because \text{ addition}$$

$$\rho = \frac{9.5}{\sqrt{9.5 \times 10}} \quad \because \text{ Given 4 with the three sums above}$$

$$= \frac{9.5}{\sqrt{95}} \quad \because \text{ multiplication}$$

$$\approx 0.975 \quad \because \ \sqrt{95} = 9.7468\ldots, \text{ division, rounded to 3 d.p.}$$

$\therefore$ With the tie, $\rho = 0.975$. If seed 0's event were a little later than seed 3's, the ranks would be $(4, 1, 2, 3, 5)$ and $\rho = 1$; if a little earlier, seeds 0 and 3 would swap and $\rho = 0.9$. So a tie gives a value between the two, outside the steps of Section 5.3. Ties change the answer, which is why they matter.

$$\boxed{\rho = 0.975 \text{ (3 d.p.)}}$$

Self-check: Which one would seed 4 distort, and why? (Pearson's C on raw values: $G_4 = 23747$ is far above the others, so it dominates the sums in **N39**.)

Likely viva question: "Your Pearson values look strong; why don't you trust them?" (With five seeds, one extreme seed can decide Pearson's value, even its sign. On ranks, that seed can move the result by at most its rank.)

5.3 Only 21 possible values with 5 seeds

In plain words: With only five seeds, ρ can't take just any value between -1 and 1. It moves in steps of 0.1. That sounds like a technical detail, but it changes how you should read the PASS threshold: with five seeds, "$\rho \geq 0.9$" means "the order is perfect, or exactly two neighbouring seeds are swapped", and nothing in between. This section derives the steps and shows what a single swap costs.

Definition: With $n = 5$ and no ties, $\rho = 1 - \sum d_i^2/20$, and $\sum d_i^2$ is an even number from 0 to 40. So ρ can only be $-1, -0.9, \ldots, 0.9, 1$: 21 values, in steps of 0.1.

Why it matters and how to read it: $\rho \geq 0.9$ means "perfect order, or exactly one adjacent swap" (**N41**). It also explains the coded cutoff 0.8999999999999998 (**N43**): it keeps a ρ of exactly 0.9 from failing through floating-point rounding. The criterion-validation record in `context.md` says ρ has only 11 values in steps of 0.2; that's wrong, and the list it gives leaves out 0.9.

The derivation has three parts. First, with $n = 5$ the general formula simplifies to $\rho = 1 - D/20$, so ρ is fixed by one whole number, D. Second, D is always even, so ρ changes in steps of $2/20 = 0.1$. Third, D is at most 40, reached when the order is fully reversed, which gives $\rho = -1$. Then the threshold can be read directly: $\rho \geq 0.9$ means $D \leq 2$, and the only orderings with $D = 0$ or $D = 2$ are the perfect order and a single swap of neighbours.

Worked example: The derivation finds the formula for $n = 5$, why $\sum d_i^2$ is even, its largest value, and the ρ of one adjacent swap.

Derivation (H.K. Dass layout): the possible values of ρ with five seeds

Given:

1. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$, no ties (**N40**).
2. $n = 5$, and each list of ranks is $1, 2, 3, 4, 5$ in some order (**N38**).
3. $d_i = R(G_i) - R(P_i)$ is an integer (**N40**).

Let:

1. D: the sum of squared rank differences, $D = \sum_i d_i^2$. Integer ≥ 0. Role: the only thing ρ depends on.
2. D_{max}: the largest possible D. Integer.
3. ρ_{swap}: the ρ when exactly one adjacent pair is swapped. Scalar.

To find: The formula for ρ at $n = 5$, the possible D, and ρ_{swap}.

Solution:

$$\rho = 1 - \frac{6D}{5(5^2-1)} \qquad \because \text{ Given 1 with } n = 5 \text{ (Given 2) and Let 1}$$

$$= 1 - \frac{6D}{5 \times 24} \qquad \because \ 5^2 - 1 = 24$$

$$= 1 - \frac{6D}{120} \qquad \because \ 5 \times 24 = 120$$

$$= 1 - \frac{D}{20} \qquad \because \text{ dividing top and bottom by 6}$$

$$\sum_i d_i = \sum_i R(G_i) - \sum_i R(P_i) \qquad \because \text{ Given 3, summed over the 5 seeds}$$

$$= 15 - 15 \qquad \because \text{ each rank list adds up to } 1+2+3+4+5 = 15 \text{ (Given 2)}$$

$$= 0 \qquad \because \text{ subtraction}$$

$$d_i^2 - d_i = d_i(d_i - 1) \qquad \because \text{ factoring out } d_i$$

$$D - \sum_i d_i = \sum_i d_i(d_i - 1) \qquad \because \text{ summing the line above over the 5 seeds}$$

$$D - 0 = \sum_i d_i(d_i - 1) \qquad \because \sum_i d_i = 0, \text{ shown above}$$

$$D = \text{an even number} \qquad \because d_i(d_i - 1) \text{ is a product of two consecutive integers, so even}$$

$$D_{\max} = 4^2 + 2^2 + 0^2 + (-2)^2 + (-4)^2 \qquad \because \text{ fully reversed order, } d = (4, 2, 0, -2, -4)$$

$$= 16 + 4 + 0 + 4 + 16 \qquad \because \text{ squaring}$$

$$= 40 \qquad \because \text{ addition}$$

$$\#\{0, 2, 4, \ldots, 40\} = \frac{40}{2} + 1 \qquad \because \text{ even numbers from 0 to 40}$$

$$= 21 \qquad \because \text{ arithmetic}$$

$$\rho_{\text{swap}} = 1 - \frac{1^2 + (-1)^2}{20} \qquad \because \text{ one adjacent swap: two seeds have } d_i = \pm 1, \text{ the rest 0}$$

$$= 1 - \frac{2}{20} \qquad \because 1^2 = (-1)^2 = 1, \text{ addition}$$

$$= 0.9 \qquad \because 2/20 = 0.1, \text{ subtraction}$$

$\therefore$ With five seeds $\rho = 1 - D/20$ with D even from 0 to 40; listing all 120 orderings shows every even value occurs, so ρ has 21 possible values. One adjacent swap gives exactly 0.9; two separate adjacent swaps give $D = 4$ and $\rho = 0.8$, a fail. (List A in Section 5.1 has $D = 14$, so $\rho = 1 - 14/20 = 0.3$, the same answer.)

$$\boxed{\rho = 1 - \frac{D}{20}, \quad 21 \text{ values}, \quad \rho_{\text{swap}} = 0.9}$$

Self-check: Can $\rho = 0.85$ happen with 5 seeds and no ties? (No: $1 - D/20 = 0.85$ needs $D = 3$, which is odd, and D is always even.)

Likely viva question: "Isn't the ρ test very coarse with five seeds?"

5.4 Significance with small n

In plain words: Even an event list that has nothing to do with grok will sometimes put the seeds in the right order, just by luck. With five seeds there are only 120 possible orders, so luck isn't rare. This section asks how often pure chance would give a ρ as high as the PASS threshold. The answer tells you how much a single PASS from five seeds can prove.

Definition: Significance (**N42**) asks whether a correlation could easily appear by chance, that is, if all $n!$ orderings of the events were equally likely. The p-value is the probability, by chance, of a ρ at least as large as the one observed.

Why it matters and how to read it: With $n = 5$, only $\rho = 1$, and at the edge $\rho = 0.9$, are unlikely by chance. Smaller values, such as List A's $\rho = 0.3$, are easy to get by chance.

The method is counting. "By chance" means each of the 120 orders of the five events is equally likely. You count the orders that give the ρ you care about and divide by 120. Only one order gives $\rho = 1$. Four more give exactly 0.9, one for each pair of neighbouring positions that can be swapped. So $\rho \geq 0.9$ comes from 5 of the 120 orders. The p-value of an observed ρ is this kind of share: the chance of a ρ at least as large. A small p-value means luck is an unlikely explanation; on its own it doesn't mean the warning is useful.

Worked example: The derivation computes the chance of $\rho = 1$ and of $\rho \geq 0.9$.

Derivation (H.K. Dass layout): chance probabilities of $\rho = 1$ and $\rho \geq 0.9$

Given:

1. $n = 5$; by chance, all $n!$ orderings are equally likely (**N42**).
2. $\rho = 1$ only for the one ordering with $D = 0$; $\rho = 0.9$ only for orderings with exactly one adjacent swap (Section 5.3).
3. With 5 positions there are 4 adjacent pairs: (1,2), (2,3), (3,4), (4,5).

Let:

1. $n!$: the number of orderings of 5 seeds. Integer.
2. $\Pr(\rho = 1)$: chance probability of $\rho = 1$. Scalar in $[0, 1]$.
3. $\Pr(\rho \geq 0.9)$: chance probability of $\rho \geq 0.9$, the p-value of $\rho = 0.9$. Scalar in $[0, 1]$.

To find: $\Pr(\rho = 1)$ and $\Pr(\rho \geq 0.9)$.

Solution:

$$
\begin{aligned}
n! &= 5 \times 4 \times 3 \times 2 \times 1 && \because \text{ definition of } n! \text{ with } n = 5 \\
&= 120 && \because \text{ multiplication} \\
\Pr(\rho = 1) &= \frac{1}{120} && \because \text{ one ordering out of 120 (Given 1, 2)} \\
\Pr(\rho \geq 0.9) &= \frac{1 + 4}{120} && \because \text{ the perfect order plus 4 single adjacent swaps (Given 2, 3)} \\
&= \frac{5}{120} && \because \text{ addition} \\
&= \frac{1}{24} && \because \text{ dividing top and bottom by 5} \\
&\approx 0.042 && \because \text{ division, rounded to 3 d.p.}
\end{aligned}
$$

$\therefore$ A random event list would reach $\rho = 1$ once in 120 tries, and $\rho \geq 0.9$ about once in 24.

$$\boxed{\Pr(\rho = 1) = \tfrac{1}{120}, \quad \Pr(\rho \geq 0.9) = \tfrac{1}{24}}$$

Reading the answer: About 1 in 24 is roughly 4%. That's small, but not tiny: if you scored 24 event lists with no link to grok at all, you'd expect about one of them to reach $\rho \geq 0.9$. Section 7.3 shows how quickly this adds up when many versions of a rule are tried.

Self-check: Why does your protocol use a threshold rather than a p-value? (With 120 orderings the p-values are coarse too; a fixed threshold is simpler to pre-register. By the same count, 14 of the 120 orderings give $\rho \geq 0.7$, a p-value of about 0.12.)

Likely viva question: "How confident can you be in a PASS from five points?"

Module 6

The frozen protocol and the PASS rule

Modules 3 to 5 built the measurements one at a time. This module puts them together into the fixed test that every predictor faces. Section 6.1 says what is fixed and when it was fixed; Section 6.2 states the PASS rule and scores two illustrative lists with it; Section 6.3 shows how the test itself is checked, by feeding it a made-up predictor that must pass.

6.1 The frozen protocol

In plain words: A test is only fair if it's the same for everyone and is decided before the results are in. The frozen protocol is your benchmark's fixed test: the same task, seeds, epochs, grok threshold and PASS rule for every predictor, written down on one date and not changed afterwards. This section says what's in it and why its date matters.

Definition: The frozen protocol (**N44**) is the fixed set of rules used to score every predictor: the config `nanda_unified.yaml`, $p = 113$, 5 seeds, 40,000 epochs, grok threshold 0.9, and the PASS rule of Section 6.2. It's written down and not changed while predictors are scored.

Why it matters and how to read it: It makes all predictors comparable, and it stops thresholds being tuned to make a favourite predictor pass. It was written on 29 September 2026. Any result that was already known by that date weakens the claim that the thresholds were chosen blind, so the dates in `context.md` matter.

The word "frozen" is the important part. If a threshold could be adjusted after seeing a predictor's numbers, it would be easy to move it until a favourite predictor passed, and the PASS would mean nothing. Freezing protects against that, but only for results that came after the freeze; that's why an examiner will ask what was already known on 29 September 2026. The protocol is applied in code, by one function, so every predictor is scored by the same lines. A verdict that the function produced and saved can be checked by anyone; one that was never saved can't.

Worked example: The verdict function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies the three conditions of Section 6.2. There's no calculation here.

Self-check: Does every predictor have a saved frozen-protocol verdict in a results file, or were some only recomputed later? (Check the saved results; a verdict that was never saved can't be checked by an examiner.)

Likely viva question: "Was the protocol fixed before you saw any results?"

6.2 The PASS rule

In plain words: The PASS rule combines the three measurements of Modules 4 and 5 into one verdict. A predictor passes only if it is early enough in enough seeds (the lead fraction), follows the order of grok (Spearman's ρ), and is early by a sensible amount, neither too late nor absurdly early (the mean relative lead). All three must hold; failing any one is a FAIL. This section summarises List A's verdict, then scores List C, a list built to look like a fixed clock.

Definition: A predictor passes (**N43**) only if all three hold:

$$\text{lead_frac} \geq 0.8, \qquad \rho \geq 0.9, \qquad 0.05 \leq \bar{r} \leq 0.9.$$

Otherwise its verdict is FAIL.

Why it matters and how to read it: Each condition blocks a different failure: lead_frac blocks firing late, ρ blocks firing on a fixed clock, and the upper limit on $\bar{r}$ blocks firing absurdly early.

Think of each condition as a check against a different way a warning can be useless. A rule that fires late, or not early enough, in more than one seed is caught by the lead fraction. A rule that fires at about the same epoch in every run, and so says nothing about when grok comes, is caught by ρ. A rule that fires so early it's probably unrelated to grok is caught by the upper limit on $\bar{r}$, and one that on average fires only just before grok is caught by the lower limit. Because the conditions catch different failures, passing two of them is no comfort: List A and List C below each pass two and fail one.

Worked example: List A (Section 4.1): lead_frac $= 0.8$ (passes, Section 4.3), $\rho = 0.3$ (fails, Section 5.1), $\bar{r} = 0.547$ (passes, Section 4.4), so FAIL. One failed condition is enough. The derivation scores a second illustrative list. Suppose a rule fires at the epochs of **List C** (**N32**), all close to epoch 2000:

$$\text{List C:} \quad (P_0, P_1, P_2, P_3, P_4) = (2020,\ 1990,\ 2000,\ 2030,\ 2010).$$

Derivation (H.K. Dass layout): the verdict of List C

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ and $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (**N26**, Section 5.1).
2. $(P_0, \ldots, P_4) = (2020, 1990, 2000, 2030, 2010)$, all five fire (List C, above).
3. Every P_i is at most 2030 and every $G_i - 100$ is at least $6921 - 100 = 6821$, so all five seeds lead (**N34**).
4. $\rho = 1 - D/20$ for $n = 5$ with no ties (Section 5.3).
5. $\bar{r} = 1 - \frac{1}{5}\sum_{i=0}^{4} P_i/G_i$ when $n = 5$ (Section 4.4).
6. PASS rule (**N43**).

Let:

1. $R(P_i)$: rank of P_i among the five events. Integer from 1 to 5. $R(P)$ is the list $(R(P_0), \ldots, R(P_4))$.

2. $d_i = R(G_i) - R(P_i)$: rank difference of seed i. Integer. d is the list $(d_0, \ldots, d_4)$.
3. $D = \sum d_i^2$. Integer.
4. lead_frac, ρ, $\bar{r}$: the three PASS quantities of List C. Scalars.
5. S: the sum $\sum_{i=0}^{4} P_i/G_i$ for List C. Scalar, positive.

To find: The three PASS quantities and the verdict.

Solution:

$$\begin{aligned}
\text{lead_frac} &= \frac{5}{5} && \because \text{ Given 3, } n = 5 \\
&= 1.0 && \because \text{ division} \\
\text{sorted } P &: 1990 < 2000 < 2010 < 2020 < 2030 && \because \text{ sorting Given 2: seeds 1, 2, 4, 0, 3} \\
R(P) &= (4, 1, 2, 5, 3) && \because \text{ position of each seed in the line above} \\
d &= (4-4,\ 1-1,\ 2-2,\ 3-5,\ 5-3) && \because \text{ Let 2 with Given 1} \\
&= (0, 0, 0, -2, 2) && \because \text{ subtraction} \\
D &= 0^2 + 0^2 + 0^2 + (-2)^2 + 2^2 && \because \text{ Let 3} \\
&= 0 + 0 + 0 + 4 + 4 && \because \text{ squaring} \\
&= 8 && \because \text{ addition} \\
\rho &= 1 - \frac{8}{20} && \because \text{ Given 4} \\
&= 0.6 && \because 8/20 = 0.4\text{, subtraction} \\
\bar{r} &= 1 - \frac{S}{5} && \because \text{ Given 5 and Let 5} \\
S &= \tfrac{2020}{12987} + \tfrac{1990}{6921} + \tfrac{2000}{9510} + \tfrac{2030}{11019} + \tfrac{2010}{23747} && \because \text{ Let 5 with Given 1 and 2} \\
&\approx 0.15554 + 0.28753 + 0.21030 + 0.18423 + 0.08464 && \because \text{ each fraction rounded to 5 d.p.; later digits dropped} \\
&= 0.92224 && \because \text{ addition} \\
\bar{r} &= 1 - \frac{0.92224}{5} && \because \text{ substitution of } S \text{ into the line } \bar{r} = 1 - S/5 \\
&= 1 - 0.184448 && \because \text{ division by 5} \\
&= 0.815552 && \because \text{ subtraction}
\end{aligned}$$

$\therefore$ List C has lead_frac $= 1.0$ (passes), $\rho = 0.6$ (fails) and $\bar{r} = 0.816$ to 3 d.p. (passes). One failed condition is enough, so the verdict is FAIL. List C fires at about epoch 2000 in every seed whatever G_i is: it's close to a fixed clock, and its ρ of 0.6 comes only from the order of differences of a few tens of epochs.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 0.6,\ \bar{r} = 0.816 :\ \text{FAIL}}$$

Reading the answer: List C shows the fixed-clock failure. Its events span only 40 epochs (1,990 to 2,030) while the grok epochs span 16,826. Its lead fraction is perfect and its mean relative lead is inside the range, so two conditions are met, but ρ catches it.

Self-check: Build a made-up event list that passes all three for your five G_i. (Section 6.3 does this with $P_i = 0.5\,G_i$.)

Likely viva question: "Justify each threshold. Where do 0.8, 0.9, 0.05 and 0.9 come from?"

6.3 Pre-registration and criterion validation

In plain words: Before trusting a test, you should check that it can tell good from bad. One way is to feed it made-up predictors whose quality you know in advance: one built to be perfect, which should pass, and one built to be useless, which should fail. If the perfect one fails, the test is too strict; if the useless one passes, it's too loose. This section does that check for the PASS rule, and explains why writing the test down before running it matters.

Definition: Pre-registration (**N46**) is writing down your test before running it. Criterion validation is checking that the test passes a good synthetic predictor and fails a bad one, where a synthetic predictor is a made-up list of event epochs built from your G_i.

Why it matters and how to read it: Your `context.md` records a calibration of the ρ cutoff. It used this run's own grok epochs, after some results were already known, so an examiner can call it post-hoc.

These two ideas answer different worries. Pre-registration answers "did you choose the test to suit the results?". Criterion validation answers "can this test pass anything at all, and does it fail what it should?". The synthetic predictor $P_i = 0.5\,G_i$ is built from the grok epochs themselves, so it follows their order exactly and always leaves half the time to grok as warning: it's the clearest case of a good warning. No real predictor could produce it, because it needs G_i in advance; its only job is to test the test.

Worked example: The derivation scores the synthetic "perfect" predictor $P_i = 0.5\,G_i$.

Derivation (H.K. Dass layout): the synthetic predictor $P_i = 0.5\,G_i$

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$, all different (**N26**).
2. Synthetic events $P_i = 0.5\,G_i$ for every seed, so all five fire.
3. Lead: $P_i < G_i - 100$ (**N34**); $r_i = (G_i - P_i)/G_i$ (**N37**).
4. PASS rule (**N43**).

Let:

1. P_i: synthetic event epoch of seed i. Scalar epoch.
2. r_i: its relative lead. Scalar.
3. lead_frac, ρ, $\bar{r}$: the three PASS quantities. Scalars.

To find: The three PASS quantities and the verdict.

Solution:

$$
\begin{aligned}
G_i - P_i &= G_i - 0.5\,G_i && \because \text{Given 2} \\
&= 0.5\,G_i && \because \text{subtraction} \\
&\geq 0.5 \times 6921 && \because \text{the smallest } G_i \text{ is 6921 (Given 1)}
\end{aligned}
$$

$$
\begin{aligned}
&= 3460.5 && \because \text{ multiplication} \\
&> 100 && \because \text{ comparison, so } P_i < G_i - 100 \text{ in every seed} \\
\text{lead_frac} &= \frac{5}{5} && \because \text{ all five seeds lead (Given 3, line above)} \\
&= 1.0 && \because \text{ division} \\
R(P_i) &= R(G_i) && \because \ P_i = 0.5\,G_i \text{ keeps the order of the } G_i \\
d_i &= 0 && \because \ d_i = R(G_i) - R(P_i) \\
\rho &= 1 - \frac{0}{20} && \because \text{ Section 5.3 with } D = 0 \\
&= 1 && \because \text{ arithmetic} \\
r_i &= \frac{0.5\,G_i}{G_i} && \because \text{ Given 3 and the second line} \\
&= 0.5 && \because \ G_i/G_i = 1 \\
\bar{r} &= \frac{5 \times 0.5}{5} && \because \text{ all five } r_i \text{ equal 0.5} \\
&= 0.5 && \because \text{ division}
\end{aligned}
$$

$\therefore$ The perfect synthetic predictor gets lead_frac $= 1.0$, $\rho = 1$ and $\bar{r} = 0.5$, so it must PASS. A synthetic event at a fixed epoch, e.g. $P_i = 2000$ in every seed, has all events tied, so ρ is undefined (Section 5.1) and can't reach 0.9; it must FAIL.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 1,\ \bar{r} = 0.5 :\ \text{PASS}}$$

Self-check: What would a synthetic control that should fail look like? (The fixed $P_i = 2000$ above; or an event placed after grok, $P_i = G_i + 100$, which has no leads.)

Likely viva question: "How do you know your criterion isn't too strict to pass anything?" (The perfect synthetic predictor passes it.)

Module 7

Research honesty

A predictor can pass the PASS rule and still not be a fair result. This module covers the ways that can happen, in the order you'd check them: a rule that uses information from the future (Section 7.1), smoothing that looks ahead (Section 7.2), trying many versions of a rule and keeping the best (Section 7.3), and a verdict that hangs on one decimal (Section 7.4). It then covers what a fair comparison needs, controls and a noise floor (Section 7.5), and ends with why a careful negative result still counts (Section 7.6).

7.1 Hindsight (look-ahead) bias

In plain words: A warning is only a warning if you could have acted on it at the time. Suppose a rule, to decide whether its event has happened at epoch 3,000, needs to know what the signal does at epoch 30,000. That rule couldn't have been run during training: at epoch 3,000 the later values don't exist yet. Such a rule has look-ahead, or hindsight. It can look excellent on paper and still be useless. This section shows how to spot it, using a simple made-up rule that compares the signal with its whole-run average.

Definition: Look-ahead bias (**N47**) is using information from later epochs, such as the final value y_{final} of a signal at epoch 40,000 (**N27**), to decide when an earlier event happened.

Why it matters and how to read it: A rule with look-ahead can pass the PASS rule and still be useless, because a live monitor couldn't run it during training: the information it needs doesn't exist yet. Any rule that uses y_{final}, the run's largest or smallest value, or an average over the whole run, has this problem, so such a result can only be reported as a hindsight check, not as a benchmark result.

The test is to imagine running the rule live: at each epoch, give it only the values up to that epoch and see whether it can decide. If it needs any value from after the epoch it's deciding about, it looks ahead. Look-ahead is easy to hide inside a single number. The final value y_{final}, the run's largest or smallest value, and the whole-run average each look like one harmless constant, but each is computed from the whole curve, including the end. The derivation makes that visible by splitting the whole-run average into a part from before the candidate event and a part from after it.

Worked example: Suppose a rule fires at the first epoch where the signal falls below its average over the whole run. The derivation shows that this average needs epochs after the event.

Derivation (H.K. Dass layout): a whole-run average needs future epochs

Given:

1. A signal y_t, recorded here at every epoch $t = 0, 1, \ldots, 40{,}000$, so $40{,}001$ values (**N27**, **N9**; illustrative).

2. The rule compares y_t with the whole-run average $\bar{y}_{\text{run}}$ and fires at the first epoch where $y_t < \bar{y}_{\text{run}}$ (illustrative rule).
3. The candidate event epoch τ is before the end of the run: $\tau < 40{,}000$.

Let:

1. y_t: the signal at epoch t. Scalar.
2. $\bar{y}_{\text{run}}$: the whole-run average, $\bar{y}_{\text{run}} = \frac{1}{40001}\sum_{t=0}^{40000} y_t$. Scalar. Role: the level the rule compares against.
3. τ: an epoch at which the rule might fire. Integer epoch.

To find: $\bar{y}_{\text{run}}$ split into a past part (epochs up to τ) and a future part (epochs after τ).

Solution:

$$\begin{aligned}
\bar{y}_{\text{run}} &= \frac{1}{40001}\sum_{t=0}^{40000} y_t && \because \text{ definition of } \bar{y}_{\text{run}} \text{ (Let 2)} \\
&= \frac{1}{40001}\left(\sum_{t=0}^{\tau} y_t + \sum_{t=\tau+1}^{40000} y_t\right) && \because \text{ splitting the sum at } \tau \\
&= \frac{1}{40001}\sum_{t=0}^{\tau} y_t + \frac{1}{40001}\sum_{t=\tau+1}^{40000} y_t && \because \text{ multiplying out } \tfrac{1}{40001}
\end{aligned}$$

$\therefore$ The second term adds up the $40{,}000 - \tau$ values after τ, and $40{,}000 - \tau \geq 1$ by Given 3. So $\bar{y}_{\text{run}}$ isn't known at epoch τ: the rule looks ahead. For $\tau = 3000$, 37,000 of the 40,001 values are from the future.

$$\boxed{\bar{y}_{\text{run}} \text{ uses } y_{\tau+1}, \ldots, y_{40000}, \text{ which come after } \tau}$$

Reading the answer: The second term is never empty, because the candidate event is before the end of the run. So at the moment the rule claims to fire, it's using numbers that don't exist yet.

Self-check: How would you test whether a rule is look-ahead free? (Cut the curve at epoch t, rerun the rule, and check that the event before t doesn't move.)

Likely viva question: "Could this signal be used during training, before you know the outcome?"

7.2 Causal vs centred moving average

In plain words: Signal curves are often noisy, so rules usually smooth them first by averaging each point with its neighbours. The question is which neighbours. If you average a point with the ones before it, the smoothed curve uses only the past. If you average it with neighbours on both sides, each smoothed point already contains a little of the future. That small difference decides whether a rule can be run live. This section writes both kinds out for a window of 3 points.

Definition: A causal moving average at epoch t (**N28**) uses only points at or before t. A centred one uses points on both sides of t.

Why it matters and how to read it: A centred average includes later epochs in each smoothed value, so any event found on a centred curve is slightly look-ahead. `uniform_filter1d` is centred (with the edges padded by copying). A rule that smooths with it can't be called an ex-ante trigger (one that uses only the past).

The look-ahead in a centred average is small but real: each smoothed value uses about half a window of points from after it, so the look-ahead grows with the window w. A causal average pays for its honesty with a delay: because it only sees the past, it reacts to a change only after the change has started, so events found on a causal curve tend to come later than on a centred one. Before you call a rule an ex-ante trigger, you need to know which kind of average it smooths with.

Worked example: The derivation writes both averages for a window of 3 points.

Derivation (H.K. Dass layout): causal and centred averages, window 3

Given:

1. A signal y_t, one scalar per point t (**N27**).
2. Window $w = 3$; causal uses points $\leq t$, centred uses both sides (**N28**).

Let:

1. y_t: the signal at point t. Scalar.
2. $\bar{y}_t^{\text{causal}}$: the causal average at t. Scalar. Uses y_{t-2}, y_{t-1}, y_t.
3. $\bar{y}_t^{\text{centred}}$: the centred average at t. Scalar. Uses y_{t-1}, y_t, y_{t+1}.

To find: Both averages, and which needs a later point.

Solution:

$$\bar{y}_t^{\text{causal}} = \frac{y_{t-2} + y_{t-1} + y_t}{3} \qquad \because \text{ the 3 points ending at } t \text{ (Given 2)}$$

$$\bar{y}_t^{\text{centred}} = \frac{y_{t-1} + y_t + y_{t+1}}{3} \qquad \because \text{ one point each side of } t \text{ (Given 2)}$$

$\therefore$ The centred average at t contains y_{t+1}, a point from the future; the causal one doesn't.

$$\boxed{\bar{y}_t^{\text{centred}} \text{ needs } y_{t+1};\ \bar{y}_t^{\text{causal}} \text{ doesn't}}$$

Self-check: Which one shifts events later? (The causal one: it only reacts after the change has happened, so its events come later than the centred version's.)

Likely viva question: "Is your smoothing causal?" (Check which function each rule uses; `uniform_filter1d` is centred.)

7.3 Researcher degrees of freedom

In plain words: Every rule has settings: a window, a threshold, how many early epochs to skip. If you try several settings and keep the one that scores best, you've given luck several chances. Section 5.4 showed that an event list with no link to grok

reaches $\rho \geq 0.9$ about once in 24 tries. This section shows what happens to that chance when you try 12 versions of a rule and keep the best.

Definition: Researcher degrees of freedom (**N48**) are choices of rule settings (**N30**), such as windows, thresholds or `skip_epochs`, made after looking at the results.

Why it matters and how to read it: Each extra version of a rule that you try is another chance to pass by luck. If settings were changed after looking at the curves, an examiner can say the rule was fitted to the data.

The danger isn't in trying things; it's in trying things after looking at the results and then reporting only the best. If the settings are chosen and written down before the curves are seen, each rule gets one chance. If they're tuned on the curves, the reported rule has had many chances, and its PASS is much weaker evidence than it looks. The calculation below treats the 12 versions as independent. They usually aren't (nearby settings tend to give similar events), so the real chance is usually lower than the answer; what the calculation shows is the direction: more versions tried means more chance of a lucky pass.

Worked example: Suppose you try 3 windows, 2 thresholds and 2 `skip_epochs` values for one signal, and keep whichever version looks best. The derivation counts the versions and estimates the chance that at least one reaches $\rho \geq 0.9$ by luck alone, treating the versions as independent. They usually aren't, so read the result as a rough picture, not an exact value.

Derivation (H.K. Dass layout): the chance that at least one version reaches $\rho \geq 0.9$ by luck

Given:

1. 3 windows, 2 thresholds, 2 `skip_epochs` values (illustrative).
2. By chance, one version reaches $\rho \geq 0.9$ with probability $\frac{1}{24}$ (Section 5.4).
3. The versions are treated as independent (simplifying assumption).

Let:

1. V: the number of versions tried. Integer. Role: how many chances there are.
2. $\Pr_{\text{none}}$: chance that no version reaches $\rho \geq 0.9$. Scalar in $[0, 1]$.
3. $\Pr_{\text{any}}$: chance that at least one version does. Scalar in $[0, 1]$.

To find: V and $\Pr_{\text{any}}$.

Solution:

$$
\begin{aligned}
V &= 3 \times 2 \times 2 && \because \text{ every combination of the settings in Given 1} \\
&= 12 && \because \text{ multiplication} \\
\Pr_{\text{none}} &= \left(1 - \tfrac{1}{24}\right)^{12} && \because \text{ each of the 12 versions misses with probability } 1 - \tfrac{1}{24} \text{ (Given 2, 3)} \\
&= \left(\tfrac{23}{24}\right)^{12} && \because 1 - \tfrac{1}{24} = \tfrac{23}{24}
\end{aligned}
$$

$$
\begin{aligned}
\Pr_{\text{any}} &= 1 - \left(\tfrac{23}{24}\right)^{12} && \because \text{“at least one” is the opposite of “none”} \\
&\approx 1 - 0.600 && \because (23/24)^{12} = 0.6000\ldots, \text{ rounded to 3 d.p.} \\
&= 0.400 && \because \text{subtraction}
\end{aligned}
$$

$\therefore$ One version has about a 4% chance of reaching $\rho \geq 0.9$ by luck; picking the best of 12 raises that to about 40%. That's why settings must be fixed before you look.

$$\boxed{V = 12, \quad \Pr_{\text{any}} \approx 0.400}$$

Reading the answer: Twelve versions isn't many: three windows, two thresholds and two skip values. Yet the chance of at least one lucky $\rho \geq 0.9$ goes from about 4% to about 40%, roughly ten times as large.

Self-check: Which settings in your config say “not from a paper”? (Check each window and skip value in `nanda_unified.yaml` for a citation.)

Likely viva question: “How many rules did you try before this one?”

7.4 Sensitivity analysis

In plain words: Every threshold in the PASS rule is a fixed number, and a predictor can land just on the wrong side of one. When that happens you want to know whether the verdict is solid or whether it would flip if the threshold, or one of the rule's settings, moved a little. Changing one thing slightly and seeing whether the verdict changes is sensitivity analysis. This section works through a made-up near-miss.

Definition: Sensitivity analysis (**N49**) is changing one setting a little and checking whether the verdict changes.

Why it matters and how to read it: If a verdict flips when a threshold or window moves a little, it's fragile. Near-misses show where that can happen.

Sensitivity analysis works in both directions. For a FAIL that misses by a little, it shows how close the predictor came. That's honest to report, but it doesn't turn the FAIL into a PASS: the thresholds are frozen. For a PASS, it shows whether the pass survives small changes; a PASS that disappears when a window changes slightly is much weaker than one that holds across a range of windows. Vary first the setting or threshold the result is closest to.

Worked example: Suppose a rule fires at $P_i = 0.09\,G_i$ in every seed (an illustrative list, **N32**). All five seeds lead and the order of the G_i is kept, so lead_frac $= 1.0$ and $\rho = 1$, as in Section 6.3. The derivation measures how far it misses the upper limit on $\bar{r}$.

Derivation (H.K. Dass layout): how far the list $P_i = 0.09\,G_i$ misses

Given:

1. $P_i = 0.09\,G_i$ for every seed (illustrative list above).

2. $r_i = (G_i - P_i)/G_i$ and $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$ (**N37**), with $n = 5$.
3. Upper limit on $\bar{r}$: 0.9 (**N43**).

Let:

1. r_i: the relative lead of seed i. Scalar.
2. $\bar{r}$: the mean relative lead of this list. Scalar.
3. δ: the miss, $\delta = \bar{r} - 0.9$. Scalar; positive means above the limit.

To find: δ.

Solution:

$$
\begin{aligned}
r_i &= \frac{G_i - 0.09\,G_i}{G_i} && \because \text{ Given 2 with Given 1} \\
&= \frac{0.91\,G_i}{G_i} && \because \text{ subtraction} \\
&= 0.91 && \because \ G_i/G_i = 1 \\
\bar{r} &= \frac{5 \times 0.91}{5} && \because \text{ all five } r_i \text{ equal 0.91 (Given 2)} \\
&= 0.91 && \because \text{ division} \\
\delta &= \bar{r} - 0.9 && \because \text{ definition of } \delta \text{ (Let 3)} \\
&= 0.91 - 0.9 && \because \text{ substitution of the line above} \\
&= 0.01 && \because \text{ subtraction}
\end{aligned}
$$

$\therefore$ This list fails only by 0.01. An upper limit of 0.92 instead of 0.9 would turn it into a PASS. A verdict that hangs on the second decimal of a threshold needs a sensitivity check.

$$\boxed{\delta = 0.01}$$

Reading the answer: This list is a perfect warning on two counts and fails the third by one hundredth: a near-miss. The honest report is FAIL, with a note that the verdict depends on the second decimal of one threshold.

Self-check: Which threshold would you vary first for a near-miss like this? (The one it misses by the least: here the upper limit 0.9 on $\bar{r}$.)

Likely viva question: "Would your verdict survive a different window size?"

7.5 Baselines, controls and noise floor

In plain words: To say a signal changes because grokking is coming, you need to know what the signal does when grokking isn't coming. Without that comparison, a change could be due to the optimiser, to the steady shrinking of the weights, or just to noise. A control run supplies the comparison, and the noise floor tells you how big a change has to be before it's worth noticing. This section explains both.

Definition: A control run (**N50**) is one where grokking doesn't happen, or a setting is switched off. The noise floor is how much a signal wobbles for no reason.

Why it matters and how to read it: Without them you can't say a change in a signal is because of grokking. A noise floor is only useful if a rule actually uses it to decide when an event is real.

Section 2.4 gave one reason this is needed: weight decay shrinks the weights in every run, whether it groks or not. A control with weight decay switched off is one way to separate the two causes. The noise floor works on a smaller scale. A signal wobbles from epoch to epoch for reasons that have nothing to do with grokking; if a rule fires on a change smaller than that wobble, the event may be just noise. Measuring the wobble is only half the job; the rule also has to use it, for example by ignoring changes smaller than it.

Worked example: A run with weight decay $\lambda = 0$ (**N19**), which often doesn't grok, would show what any signal y_t does without grokking. There's no calculation here.

Self-check: How would you estimate the noise floor from the saved curves? (For example, take a stretch of a signal's curve and measure how much it moves from one point to the next; the typical size of those moves is a first estimate of the noise floor.)

Likely viva question: "Where's your control condition?"

7.6 Negative results and reproducibility

In plain words: Most of your predictors failed. That isn't the same as the thesis failing. A careful "this doesn't work", from a fair test that anyone can rerun, is a result: it tells the next researcher which ideas not to rely on. This section explains what makes a negative result worth reporting and how reproducibility supports it.

Definition: A negative result (**N51**) is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

Why it matters and how to read it: Seven of your eight predictors are closed-negative (**N45**), so the thesis's value is in showing the tests were fair and repeatable: fixed seeds, saved data, frozen thresholds.

A negative result is only as good as the test behind it. If the test was too strict, the failures say nothing; that's why criterion validation (Section 6.3) matters, because it shows that a good predictor would have passed. If the test was changed along the way, the failures can't be compared; that's why the protocol is frozen (Section 6.1). And if nobody can rerun the work, nobody can check it; that's why fixed seeds and saved data matter. Together they turn "seven predictors failed" into "seven predictors were fairly tested and failed".

Worked example: Recomputing from the saved accuracy curves gives the same G_i, and rebuilding the split with the same seed reproduces the saved accuracies. There's no calculation here.

Self-check: What makes a negative result publishable rather than just a failure?

Likely viva question: "If almost nothing passes, what is the contribution of the thesis?"

Recommended reading

[1] Power, A., Burda, Y., Edwards, H., Babuschkin, I., Misra, V. (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.* arXiv:2201.02177. (Modules 1–2.)

[2] Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J. (2023). *Progress Measures for Grokking via Mechanistic Interpretability.* ICLR 2023. (Modules 1–2; your setup.)

[3] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR 2019. (AdamW, Module 2.)

[4] Spearman, C. (1904), the original paper, or any introductory statistics chapter on Spearman rank correlation. (Module 5.)

[5] Nosek, B. A. et al. (2018). *The Preregistration Revolution.* PNAS 115(11). (Modules 6–7.)